



UNIVERSITY OF SARAJEVO
FACULTY OF ELECTRICAL ENGINEERING
DEPARTMENT OF COMPUTER SCIENCE AND INFORMATICS

Database Optimization via Machine Learning

MASTER'S THESIS
- SECOND CYCLE OF STUDY -

Student: Ajdin Šuta

Supervisor: Doc. dr Kenan Šehić

Sarajevo, September 2026



UNIVERZITET U SARAJEVU
ELEKTROTEHNIČKI FAKULTET
ODSJEK ZA RAČUNARSTVO I INFORMATIKU

Optimizacija velikih baza podataka sa mašinskim učenjem

ZAVRŠNI RAD
- DRUGI CIKLUS STUDIJA -

Student: Ajdin Šuta

Mentor: Doc. dr Kenan Šehić

Sarajevo, Septembar 2026

Sažetak

Sistemi za upravljanje bazama podataka izlažu veliki broj međusobno zavisnih konfiguracijskih parametara čiji utjecaj zavisi od radnog opterećenja, raspoloživih resursa i stanja sistema. Njihovo automatsko podešavanje je skup problem optimizacije bez eksplicitno poznate funkcije cilja, jer svaka procjena zahtijeva primjenu konfiguracije i mjerenje stvarnog sistema. U ovom radu je razvijen okvir za pouzdano i ponovljivo podešavanje PostgreSQL-a primjenom Bayesove optimizacije. Okvir automatizira obnovu kontrolisanog stanja baze, aktiviranje i provjeru konfiguracije, zagrijavanje sistema, mjerenje radnog opterećenja, prikupljanje telemetrije, povratak na sigurnu konfiguraciju i čuvanje provjerljivog traga eksperimenta.

Primarna evaluacija ispituje koliko pouzdano strategije pretrage usmjeravaju jednak logički budžet kandidata prema korisnim regijama konfiguracijskog prostora. U tom okviru poredi jednociljnu metodu qLogNEI i višeciljne metode qLogNParEGO i qLogNEHVI sa slučajnim i Sobol pretraživanjem. Optimizuju se propusnost i 99. percentil latencije u prostoru od osam PostgreSQL parametara. Primarno poređenje obuhvata jedinstven eksperiment sa pet sjemena. U završenom eksperimentu najbolje pronađene konfiguracije bile su vrlo bliske po propusnosti, dok se jasnija razlika pojavila u kvalitetu kandidata kroz cijeli budžet: Bayesove metode su u prosjeku održale propusnost blizu istovremenih kontrolnih mjerenja i češće su birale konfiguracije koje poboljšavaju oba cilja, dok su neusmjerene metode češće trošile procjene na degradirajuće regione. Višeciljne metode su proširile posmatrani Pareto front, ali rezultati ne opravdavaju proglašenje jedne akvizicijske funkcije univerzalnim pobjednikom. U odvojenoj potvrdi (F4), odabrana konfiguracija E ostvarila je prosječno 5,1% veću propusnost i 3,4 ms niži p99 od podrazumijevane konfiguracije mjerene u istom bloku, kroz četiri bloka bez kvarova. Ovaj nalaz vrijedi za ponovljena mjerenja odabrane konfiguracije, a ne za prosjek pretrage ili nadmoć akvizicijske metode iz koje je potekla. Ispitivanje kraćih mjerenja pokazalo je operativnu izvodljivost, ali samo skromnu uštedu ukupnog vremena zbog fiksnih troškova obnove, ponovnog pokretanja i zagrijavanja. Doprinos rada je zato prvenstveno u pouzdanom eksperimentalnom okviru i efikasnijem usmjeravanju ograničenog budžeta prema obećavajućim regionima, a ne u tvrdnji o apsolutno najboljoj konfiguraciji.

Ključne riječi: PostgreSQL, konfiguracija baze podataka, Bayesova optimizacija, Gaussovi procesi, višeciljna optimizacija, pouzdanost, propusnost, repna latencija

Abstract

Database management systems expose many interacting configuration parameters whose effects depend on the workload, available resources, and system state. Automatic tuning is therefore an expensive black-box optimization problem: each evaluation requires applying a configuration and measuring the running system. This thesis develops a framework for reliable and reproducible PostgreSQL configuration tuning with Bayesian optimization. The framework automates restoration of a controlled database state, configuration activation and verification, warm-up, workload measurement, telemetry collection, rollback to a safe configuration, and preservation of an auditable experiment record.

The primary evaluation examines how dependably search strategies direct equal logical candidate budgets toward useful regions of the configuration space. Within this design, the single-objective qLogNEI method and the multi-objective qLogNParEGO and qLogNEHVI methods are compared with random and Sobol search. Throughput and 99th-percentile latency are optimized over eight PostgreSQL parameters. The primary comparison comprises one five-seed experiment. The best configurations were closely matched in raw throughput, whereas a clearer distinction appeared in candidate quality across the full budget: the Bayesian methods maintained average throughput near contemporaneous default controls and more frequently selected configurations that improved both objectives, while undirected search spent more evaluations in performance-degrading regions. The multi-objective methods expanded the observed Pareto front, but the evidence does not justify naming one acquisition function as a universal winner. In separate finalist confirmation (F4), selected configuration E achieved 5.1% higher mean control-relative throughput and a 3.4 ms mean p99 reduction against the same-block default across four failure-free blocks. This finding concerns repeated evaluation of a selected configuration, rather than average search performance or superiority of its originating acquisition method. Shorter measurements were operationally feasible but yielded only modest end-to-end savings because restoration, restart, verification, and warm-up costs remained fixed. The main contribution is therefore a reliable experimental framework and a more effective allocation of a limited evaluation budget to promising regions, not a claim of a uniquely superior configuration.

Keywords: PostgreSQL, database configuration, Bayesian optimization, Gaussian processes, multi-objective optimization, reliability, throughput, tail latency

Sarajevo, Septembar 2026

Postavka zadatka završnog rada II ciklusa

Optimizacija velikih baza podataka sa mašinskim učenjem

Kratki pregled teme:

Moderne velike baze podataka posjeduju stotine konfiguracijskih parametara čije ručno podešavanje zahtijeva značajno ekspertsko znanje i vrijeme. Neadekvatna konfiguracija često rezultira smanjenim performansama i prekomjernom potrošnjom resursa, zbog čega automatizacija ovog procesa postaje neophodna. Ova teza istražuje primjenu metoda mašinskog učenja, sa primarnim fokusom na Bajesovu optimizaciju, u svrhu automatskog podešavanja parametara baza podataka. Inspirisan savremenim industrijskim rješenjima, rad analizira kako Bajesova optimizacija kreira probabilistički model sistema kako bi efikasno pretražila visokodimenzionalni prostor parametara. Cilj istraživanja je procijeniti efikasnost ovog pristupa u smanjenju vremena izvršavanja upita, optimizaciji iskorištenosti resursa i uspostavljanju potpuno automatizovanog procesa konfigurisanja za sisteme velikih razmjera.

Literatura:

- [1] Dana Van Aken, Andrew Pavlo, Geoffrey J. Gordon, and Bohan Zhang. 2017. Automatic Database Management System Tuning Through Large-scale Machine Learning. In *Proceedings of the 2017 ACM International Conference on Management of Data (SIGMOD '17)*. Association for Computing Machinery, New York, NY, USA, 1009–1024. <https://doi.org/10.1145/3035918.3064029>
- [2] Bobak Shahriari, Kevin Swersky, Ziyu Wang, Ryan P. Adams and Nando de Freitas. *Taking the Human Out of the Loop: A Review of Bayesian Optimization*.

Izjava o autentičnosti radova

Završni rad II ciklusa studija

Ime i prezime: Ajdin Šuta

Naslov rada: Optimizacija velikih baza podataka sa mašinskim učenjem

Vrsta rada: Završni rad drugog ciklusa studija

Broj stranica: 61

Potvrđujem:

- da sam pročitao dokumente koji se odnose na plagijarizam, kako je to definisano Statutom Univerziteta u Sarajevu, Etičkim kodeksom Univerziteta u Sarajevu i pravilima studiranja na Univerzitetu u Sarajevu;
- da sam svjestan univerzitetskih disciplinskih pravila koja se tiču plagijarizma;
- da je rad koji predajem moj samostalni rad, osim u dijelovima gdje je drugačije jasno naznačeno;
- da rad nije predat, u cjelini ili djelimično, za stjecanje zvanja na Univerzitetu u Sarajevu ili drugoj visokoškolskoj ustanovi;
- da sam jasno naznačio citirani i parafrazirani materijal i naveo sve korištene izvore;
- da sam dosljedno primijenio odabrani stil citiranja i naveo potpune bibliografske podatke korištenih izvora;
- da sam odgovarajuće naznačio svaku pomoć dobijenu pored pomoći mentora i akademskih tutora ili tutorica.

Sarajevo, Septembar 2026

Potpis:

Ajdin Šuta

Contents

Sažetak	i
Abstract	ii
List of Figures	viii
List of Tables	ix
List of Abbreviations	x
1 Introduction	1
1.1 Problem Context and Motivation	1
1.2 Problem Statement	2
1.3 Research Aim and Objectives	2
1.4 Research Questions	3
1.5 Scope and Delimitations	3
1.6 Contributions of the Thesis	3
1.7 Organization of the Thesis	4
2 Background and Theoretical Foundations	5
2.1 DBMS Configuration Parameters and Workload Dependence	5
2.2 PostgreSQL Performance Configuration	6
2.3 Database Performance Measurement	6
2.4 Database Tuning as Expensive Black-box Optimization	6
2.5 Gaussian-process Surrogate Models	7
2.6 Acquisition Functions and Sequential Decision-making	8
2.6.1 Improvement-based acquisitions	9
2.7 Multi-objective Bayesian Optimization	9
2.7.1 Dominance and Pareto optimality	9
2.7.2 Random scalarization and ParEGO	9
2.7.3 Expected hypervolume improvement	10
2.8 Initialization, Dimensionality, and Mixed Parameter Domains	10
2.9 Multi-fidelity Optimization	10
3 Related Work	12
3.1 Evolution of Automated and Self-tuning Database Systems	12
3.2 Bayesian-optimization-based DBMS Tuning	12
3.3 Reinforcement Learning and Alternative Approaches	13
3.4 Knob Selection and Search-space Reduction	13

3.5	Positioning of the Framework and the Research Gap	14
4	Framework Design and Implementation	15
4.1	System Requirements and Design Principles	15
4.2	Overall Architecture	15
4.3	Configuration-space Representation	16
4.4	Optimization Components	17
4.4.1	Model Fitting and Proposal Generation	17
4.5	Candidate Evaluation Lifecycle	19
4.6	Safety, Failure Handling, and Resumption	19
4.7	Evidence Integrity, Testing, and Reproducibility	20
5	Experimental Methodology	21
5.1	Research Design and Experimental Environment	21
5.2	Dataset Preparation and Candidate-level Restoration	22
5.3	Workload, Warm-up, and Duration Calibration	23
5.4	Default Controls and Search-space Finalization	24
5.5	Primary Fixed-Budget Experiment and Analysis	26
5.5.1	Budget Allocation and Shared Bayesian Initialization	26
5.5.2	Candidate Budgets, Seeds, and Temporal Interleaving	26
5.5.3	Objectives, Reference Point, and Hard Gates	27
5.5.4	Endpoints and Control-relative Analysis	27
5.5.5	Retry, Failure, Stopping, and Statistical Rules	28
5.6	Multi-fidelity Evaluation	29
5.6.1	Retrospective Prefix Evaluation	30
5.6.2	Live Promotion Demonstration	30
5.7	Confirmatory Evaluation Protocol	31
6	Experimental Results	33
6.1	Calibration and Validation Results	33
6.2	Execution Integrity and Budget Accounting	35
6.3	Fixed-budget Search Effectiveness	36
6.3.1	Candidate Quality Across the Full Budget	36
6.3.2	Best-found Performance and Frontier Development	38
6.3.3	Post-hoc Surrogate Diagnostic	40
6.3.4	Temporal Drift and Per-seed Variability	42
6.3.5	Endpoint Uncertainty and Ranking Boundaries	43
6.4	Multi-fidelity and Confirmatory Results	43
6.4.1	Retrospective Prefix Evaluation	43
6.4.2	Live Promotion and Runtime Savings	44
6.4.3	Finalist Confirmation and Configuration Selection	44
7	Discussion	46
7.1	Reliability of Measurement and Budget Accounting	46
7.2	Full-budget Effectiveness and Objective Trade-offs	47
7.3	Multi-fidelity and Operational Implications	47
7.4	Relationship to Prior Work and Practical Implications	48
7.5	Threats to Validity and Limitations	48

8 Conclusion and Future Work	50
8.1 Conclusions	50
8.2 Answers to the Research Questions	51
8.3 Limitations	51
8.4 Future Work	52
Appendices	53
A Selected PostgreSQL Configuration and Encoding	54
A.1 Confirmed Configuration and Native Units	54
A.2 Normalized Encoding and Candidate Identity	55
B Protocol, Reproduction, and Evidence Traceability	56
B.1 Software and Experimental Specification	56
B.2 Observation Lineage and Evidence Populations	57
B.3 Offline Verification and Reporting	58
Bibliography	59

List of Figures

2.1	Gaussian-process learning and candidate selection	8
2.2	Measurement fidelity and promotion cost	11
4.1	Architecture of the configuration-evaluation framework	16
5.1	Primary evaluation and permitted optimizer feedback	27
6.1	Per-seed full-budget candidate quality	37
6.2	Search trajectories across thirty logical slots	40
6.3	Physical candidate outcomes and pooled Pareto frontier	41
6.4	Interleaved default controls and drift flags	42

List of Tables

3.1	Selected model-based DBMS tuning systems	13
4.1	Candidate-generation mechanisms	17
4.2	Gaussian-process specification used in the experiments	18
5.1	Experimental environment and full-fidelity benchmark profile	22
5.2	Knob-screening evidence and final decision	25
5.3	Primary configuration space and default control values in native units	25
5.4	Logical budgets and physical observation allocation	26
5.5	Common-seed blocks and treatment order for confirmation	31
6.1	Deviations from protocol	34
6.2	Primary physical observations and infrastructure attempts	35
6.3	Primary completed-attempt time accounting	36
6.4	Full-budget candidate quality relative to local defaults	36
6.5	Joint-improvement margin sensitivity	37
6.6	Post-hoc initialization and adaptation decomposition	38
6.7	Final raw search endpoints	39
6.8	Post-hoc GP length-scale diagnostic	41
6.9	Whole-seed uncertainty for full-budget endpoints	43
6.10	Retrospective prefix results	44
6.11	Four-block finalist confirmation	45
A.1	Confirmed configuration E	54
B.1	Software and protocol entry points	56
B.2	Evidence locations relative to the external evidence root	57

List of Abbreviations

AMD64	Advanced Micro Devices 64-bit architecture
ARD	Automatic relevance determination
BO	Bayesian optimization
CPU	Central processing unit
DBMS	Database management system
EHVI	Expected hypervolume improvement
EI	Expected improvement
F2	Short-fidelity prefix measurement (60 or 120 seconds)
F3	Full-fidelity measurement (600 seconds)
F4	Four-block confirmatory evaluation
GiB	Gibibyte
GP	Gaussian process
HV	Hypervolume
I/O	Input/output
JSON	JavaScript Object Notation
L-BFGS-B	Limited-memory Broyden–Fletcher–Goldfarb–Shanno algorithm with bound constraints
LogEI	Logarithmic expected improvement
MB	Megabyte
MiB	Mebibyte
ms	Millisecond
OLTP	Online transaction processing
p99	99th percentile
ParEGO	Pareto-efficient global optimization
PID	Process identifier
qEHVI	Batch expected hypervolume improvement
qLogNEHVI	Batch logarithmic noisy expected hypervolume improvement
qLogNEI	Batch logarithmic noisy expected improvement
qLogNParEGO	Batch logarithmic noisy parallel ParEGO acquisition
qNEHVI	Batch noisy expected hypervolume improvement
RBF	Radial basis function
RL	Reinforcement learning
RQ	Research question
s	Second
SHA-256	Secure Hash Algorithm 256-bit
SMAC	Sequential Model-based Algorithm Configuration
SQL	Structured Query Language
TPC-B	Transaction Processing Performance Council Benchmark B
TPS	Transactions per second
TuRBO	Trust-region Bayesian optimization
WAL	Write-ahead log

Chapter 1

Introduction

1.1 Problem Context and Motivation

Database management systems (DBMSs) are expected to deliver predictable performance while serving workloads whose resource demands and access patterns change over time. Their behavior is controlled by many configuration parameters that govern memory allocation, write-ahead logging, checkpoint activity, query planning, parallel execution, and other subsystems. These parameters are not independent: changing one value may alter the usefulness or safe range of another. Their effects are also conditional on the workload, dataset, hardware, and software version. Consequently, a configuration that performs well in one environment cannot be assumed to transfer unchanged to another. Early automated tuning research and later machine-learning-based systems identify this interaction, workload dependence, and cost of acquiring performance evidence as central difficulties of DBMS configuration tuning [1, 3, 4].

The problem becomes more demanding when performance is described by more than one objective. Transaction throughput measures the amount of useful work completed per unit of time, but it does not reveal how slowly the least responsive transactions complete. A high-percentile latency measure, such as the 99th percentile (p99), characterizes this tail behavior. Maximizing transactions per second (TPS) and minimizing p99 latency can lead to competing preferences rather than a single natural ordering of configurations. The result is a set of trade-offs: one configuration can be preferable for throughput, another for tail latency, and neither need dominate the other.

Performance cannot be inferred reliably from parameter values alone. A candidate must be applied to a running DBMS, activated correctly, and evaluated under a controlled workload. The measured outcome may be affected by cache state, maintenance activity, background processes, temporal drift, and failures outside the optimizer. Configuration tuning is therefore both an optimization problem and an experimental-systems problem. A useful tuning method must choose informative candidates, while the surrounding system must make their measurements comparable, recoverable, and auditable.

Manual tuning depends on expert knowledge and is difficult to repeat whenever the workload or resource envelope changes. Exhaustive search is also impractical: even a moderate number of integer and continuous parameters creates a configuration space that is too large to enumerate, and every measurement consumes database and wall-clock resources. General comparative work on DBMS configuration tuning consequently treats candidate selection, parameter selection, and the reuse of prior evidence as distinct design problems [4].

Bayesian optimization (BO) is well suited to settings in which the objective is expensive to observe and derivatives are unavailable. It uses a probabilistic surrogate model to summarize

previous observations and an acquisition function to balance exploitation of regions that appear promising with exploration of regions where uncertainty remains high [2, 5]. Unlike random or space-filling sampling, BO can adapt its next proposal to the outcomes already measured. This does not guarantee that its single best configuration will be better under every finite budget. Its practical value may instead appear in how it allocates the budget: a directed method can spend more evaluations in promising regions and fewer on configurations that substantially degrade performance.

This thesis therefore considers the quality distribution across the candidate budget, performance relative to contemporaneous default controls, and failure handling alongside best-found performance and Pareto-front quality.

1.2 Problem Statement

This thesis studies automatic tuning of PostgreSQL configuration parameters as an expensive, noisy, mixed-domain, multi-objective black-box optimization problem. The optimizer receives an eight-dimensional configuration vector and proposes a candidate within predefined validity and resource-safety bounds. The candidate is evaluated on a controlled `pgbench` workload. The two main observations are TPS, which is maximized, and p99 transaction latency in milliseconds, which is minimized.

Because each valid observation requires controlled restoration, activation, measurement, and rollback, only a limited number of full-fidelity evaluations is affordable. Failures in that process must remain distinct from optimization outcomes.

The research problem is thus to determine whether model-based candidate selection uses this limited experimental budget more effectively and reliably than undirected search, without confusing optimization quality with infrastructure artifacts. The comparison includes a throughput-oriented single-objective method, two multi-objective methods, and random and Sobol baselines. It evaluates both endpoint quality and the quality of candidates encountered throughout the search. Confirmatory measurements are treated as a separate evidential stage, so small differences between finalists are not presented as settled before repeated evaluation.

1.3 Research Aim and Objectives

The aim of the thesis is to design, implement, and evaluate a reliable framework for machine-learning-assisted PostgreSQL configuration tuning under a limited measurement budget. Reliability here means more than successful software execution. It includes controlled state, verifiable activation of settings, explicit recovery behavior, durable provenance, transparent treatment of drift and failures, and conclusions that match the resolution of the evidence.

The specific objectives are to:

1. formalize PostgreSQL configuration tuning as a throughput–tail-latency optimization problem over a bounded mixed parameter space;
2. construct an auditable candidate lifecycle with controlled state, verified activation, failure handling, rollback, and durable evidence;
3. establish a fixed-budget design that preserves equal logical evaluation opportunities, isolates adaptive histories, and exposes temporal drift;
4. implement single-objective and multi-objective Gaussian-process BO and comparable random and Sobol baselines within that common framework;

5. evaluate how the methods allocate their budgets using control-relative and full-budget measures, alongside best-found and Pareto-front quality;
6. evaluate temporal drift, operational cost, and shorter prefix measurements; and
7. confirm finalist-level conclusions in a distinct evidential stage.

1.4 Research Questions

The investigation is guided by the following research questions:

- RQ1** How consistently do the compared search strategies allocate their budgets to candidates that avoid degradation relative to contemporaneous default controls, beyond the performance of the single best candidate?
- RQ2** Can the restore–restart–measure–rollback lifecycle produce complete, auditable, and reproducible evidence while keeping infrastructure failures, invalid measurements, and temporal drift visible?
- RQ3** Under the same fixed logical candidate budget, how do qLogNEI, qLogNParEGO, and qLogNEHVI compare with random sampling, Sobol sampling, and the PostgreSQL default in throughput, p99 latency, and observed Pareto-front quality?
- RQ4** Do 60- or 120-second prefix measurements preserve the ordering of promising full-fidelity candidates, and do they yield meaningful end-to-end runtime savings after fixed operational costs are included?
- RQ5** Do apparent differences between finalist configurations remain stable under a separately controlled confirmatory evaluation?

1.5 Scope and Delimitations

The empirical scope is intentionally narrow: one containerized PostgreSQL version, one synthetic transactional workload and resource envelope, and an eight-parameter space. The primary comparison is one five-seed experiment with equal logical candidate budgets. Per-seed values and effect sizes are retained because this design provides limited inferential resolution. Interleaved default controls expose temporal change, and confirmatory measurements form a distinct evidential stage.

The study does not claim that the selected eight parameters are universally the most important PostgreSQL settings. It does not optimize indexes or schema, learn a general feasibility boundary, evaluate every workload class, or test cross-system transfer. The framework is not deployed as an autonomous production controller, and measured finalists are not applied to a production system. Conclusions therefore refer to the evaluated workload, resource envelope, parameter bounds, and experiment protocol.

1.6 Contributions of the Thesis

The thesis does not introduce Bayesian optimization, Gaussian processes, or a new DBMS. Its principal contribution is a reliability-aware, fixed-budget framework for evaluating how established optimization methods use expensive PostgreSQL measurements. Reliability of the measurement process makes it possible to assess reliability of budget allocation: whether a strategy

repeatedly selects useful candidates rather than depending only on one favorable endpoint. This contribution comprises:

- an automated and recoverable measurement lifecycle with verified configuration activation, rollback, and durable evidence;
- explicit accounting of logical candidate slots, shared physical initialization, controls, and infrastructure retries so that equal-budget claims retain a precise meaning;
- a common implementation of three Bayesian methods and two undirected baselines with isolated adaptive histories;
- a protocol that evaluates control-relative quality across the full candidate budget alongside endpoint and Pareto-front quality while exposing drift and failures; and
- a staged evaluation of prefix-based decisions and finalist confirmation that accounts for full lifecycle cost and evidential uncertainty.

1.7 Organization of the Thesis

Chapter 2 provides the required database and optimization background, and Chapter 3 positions the work within automated DBMS tuning research. Chapter 4 describes the framework; Chapters 5 and 6 define the experimental method and report its results. Chapters 7 and 8 discuss the findings, answer the research questions, and state the limitations and future work.

Chapter 2

Background and Theoretical Foundations

2.1 DBMS Configuration Parameters and Workload Dependence

A database management system exposes configuration parameters that determine how it allocates resources and how its internal components behave. These parameters include memory budgets, write-ahead-log and checkpoint controls, planner cost constants, statistics targets, and limits on parallel execution. After a parameter has been associated with an admissible set of values, it is also referred to as a configuration *knob*. A complete assignment of values to the selected knobs forms a configuration vector $\mathbf{x} = (x_1, \dots, x_d)$ in a search space X .

The performance response to $\mathbf{x}$ is conditional on its operating context. Let w denote the workload, D the dataset and its physical state, h the hardware and resource envelope, and v the DBMS and software version. A performance objective can then be represented as

$$y = f(\mathbf{x}; w, D, h, v) + \varepsilon, \quad (2.1)$$

where f is unavailable in closed form and ε represents measurement variation. Equation (2.1) makes two properties explicit. First, a configuration is not intrinsically good or bad; its quality is defined for a particular context and objective. Second, repeating an apparently identical measurement need not return exactly the same value.

The response surface is difficult to search because knob effects can be nonlinear and non-monotonic. A larger memory setting may avoid temporary I/O in one workload but exhaust memory when many sessions execute memory-intensive operators concurrently. Increasing a checkpoint interval can reduce checkpoint frequency while increasing recovery work and storage requirements. A planner cost change can switch an entire execution plan rather than produce a gradual change. Interactions are equally important: the value of additional shared buffers can depend on the working set, the operating-system cache, and the rate at which dirty pages must be written. Early work on iTunes demonstrated such complex response surfaces, while subsequent systems confirmed that workload, hardware, and software context constrain the transferability of tuning results [1, 3, 6].

Configuration tuning differs from query optimization and physical database design. Query optimization selects an execution plan for a query, while physical design changes structures such as indexes, partitions, and materialized views. Configuration tuning instead changes server-level or session-level controls that influence many queries and subsystems. These activities interact, but this thesis holds the schema and physical design fixed so that the measured differences can be attributed to the selected PostgreSQL knobs under the controlled workload.

2.2 PostgreSQL Performance Configuration

PostgreSQL documents configuration parameters through server metadata and the server-configuration reference. Values must be interpreted with their units and activation contexts: some settings are session-level, whereas others require a reload or restart [7]. The framework consequently uses server-reported metadata and verifies active values after configuration changes.

The selected memory knobs have different semantics. `shared_buffers` allocates the server buffer pool, `effective_cache_size` is only a planner estimate, and `work_mem` is available to individual sort or hash operations and can multiply across operations and sessions. Safe bounds must therefore reflect both the operating-system cache and workload concurrency.

Write-ahead logging and checkpoint settings jointly control when dirty pages are written. `checkpoint_timeout` and `max_wal_size` affect checkpoint frequency, while `checkpoint_completion_target` spreads checkpoint writes across the interval. Their interaction can change throughput, latency variability, storage demand, and recovery work.

Planner and parallel-execution knobs influence plan selection rather than reserving resources directly. `random_page_cost` changes the relative attractiveness of non-sequential access, and `max_parallel_workers_per_gather` limits workers requested by eligible plans. Their effects depend on plan eligibility, system-wide limits, query duration, and contention. These semantics show why an optimization space must define units, activation rules, resource multiplicity, and interactions in addition to numerical bounds [7].

2.3 Database Performance Measurement

An optimizer can only be as reliable as its observations. Database performance measurement must specify the workload, client concurrency, dataset, run duration, warm-up behavior, metric definitions, and treatment of failures. PostgreSQL’s `pgbench` repeatedly executes a transaction script through concurrent sessions and reports a transaction rate and latency statistics [7]. These outputs become experimental evidence only after the surrounding database state and measurement procedure are controlled.

Throughput is the rate of successfully completed transactions and is comparable only when the workload, offered load, duration, and failure policy are fixed. It does not describe slow transactions. This study therefore also uses p99, the empirical latency below which approximately 99% of successful transactions complete. Tail latency can expose an operationally important minority of slow requests that a mean conceals [8].

Reliable percentiles require sufficient observations and a defined treatment of retries and failures. Raw transaction logs are therefore retained, and small differences are interpreted against run-to-run variation. CPU, memory, I/O, WAL, and database counters support diagnosis and validity checks, while TPS and p99 remain the optimization objectives [7].

2.4 Database Tuning as Expensive Black-box Optimization

For a single performance objective under the selected workload and environment, configuration tuning can be written as

$$\mathbf{x}^* \in \arg \max_{\mathbf{x} \in X_{\text{safe}}} f(\mathbf{x}), \quad (2.2)$$

where X_{safe} is the valid, resource-safe subset of the configuration domain. The function has no accessible analytic form, its derivatives are unavailable, and evaluating it requires applying a

candidate and executing a workload. It is therefore a black-box objective. The restore, restart, warm-up, measurement, verification, and rollback steps make each evaluation expensive.

The observations are noisy because the operating system, cache hierarchy, background DBMS activity, and benchmark clients introduce variation. The optimization algorithm must distinguish this observation noise from invalid evidence. A benchmark result after the wrong database restore is not a noisy sample of the intended objective; it is a failed experimental procedure. The same distinction applies when a requested setting was not activated. The framework’s lifecycle checks therefore define whether an observation is eligible for optimizer feedback.

Exhaustive evaluation is infeasible even for the eight selected knobs. If each had only ten possible levels, a full factorial design would contain 10^8 configurations. Continuous knobs and wider integer ranges make the unrounded space larger still. Random search and low-discrepancy Sobol designs are useful baselines because they do not require a learned response model, but they do not adapt their future samples to measured performance. Sequential model-based optimization instead fits a surrogate and uses it to allocate the remaining budget [2, 9].

2.5 Gaussian-process Surrogate Models

A Gaussian process (GP) defines a probability distribution over functions. It is specified by a mean function $m(\mathbf{x})$ and a covariance kernel $k(\mathbf{x}, \mathbf{x}')$:

$$f(\mathbf{x}) \sim \text{GP}(m(\mathbf{x}), k(\mathbf{x}, \mathbf{x}')). \quad (2.3)$$

The kernel expresses assumptions about how strongly function values at two configurations are related. Given observations $D_n = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$, GP regression produces a posterior predictive distribution at an unevaluated point $\mathbf{x}$. With Gaussian observation noise of variance σ_ϵ^2 , its latent mean and variance can be written as

$$\mu_n(\mathbf{x}) = m(\mathbf{x}) + \mathbf{k}_\mathbf{x}^\top (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} (\mathbf{y} - \mathbf{m}), \quad (2.4)$$

$$\sigma_n^2(\mathbf{x}) = k(\mathbf{x}, \mathbf{x}) - \mathbf{k}_\mathbf{x}^\top (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{k}_\mathbf{x}, \quad (2.5)$$

where $\mathbf{K}_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$ and $\mathbf{k}_\mathbf{x}$ contains covariances between the training inputs and $\mathbf{x}$ [10].

The posterior mean summarizes predicted performance, while the posterior variance represents model uncertainty under the GP assumptions. This dual output makes GPs useful for Bayesian optimization: the acquisition function can prefer high predicted values, high uncertainty, or a combination. The uncertainty is epistemic model uncertainty, not a guarantee that all sources of experimental error have been characterized.

Figure 2.1 illustrates this update in one normalized configuration coordinate. Before measurement, the prior permits many response curves; conditioning on observations concentrates the distribution near measured inputs while leaving larger uncertainty in sparsely sampled regions. The shaded posterior band describes the latent response under fixed model assumptions. Predicting a new noisy measurement additionally includes σ_ϵ^2 in its variance [10]. The acquisition panel shows why the next evaluation need not maximize the posterior mean: uncertainty can give an unmeasured region a substantial chance of improvement. An eight-knob surrogate applies the same conditioning principle to configuration vectors, although its response surface cannot be fully represented by this one-dimensional illustration.

GPs are attractive when evaluations are scarce, but their usefulness depends on modeling assumptions and dimensionality. A stationary global kernel may smooth over abrupt plan changes, and computational cost grows with the number of observations. Local-model methods such as TuRBO address difficult high-dimensional settings by allocating evaluations among local trust regions [11]. The present study instead reduces the domain to eight screened knobs and uses a

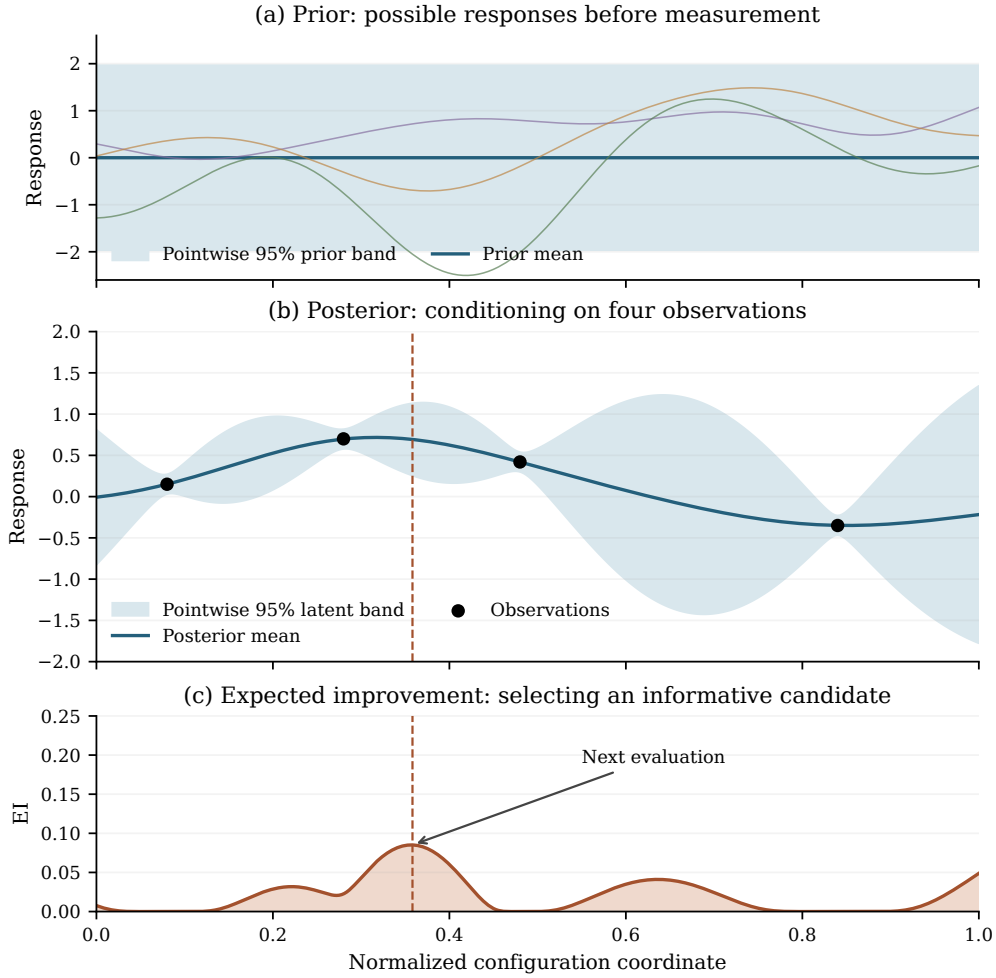


Figure 2.1: Gaussian-process learning in an original synthetic example, not database measurements. The prior and posterior use a squared-exponential kernel; the bands are pointwise 95% latent intervals, not simultaneous bounds or experimental confidence intervals. For illustration, expected improvement (EI) uses the largest observation as its incumbent. The implemented noisy acquisitions instead account for uncertainty in the incumbent.

fixed, modest candidate budget, making a global GP a reasonable object of empirical evaluation rather than an assumed perfect model.

2.6 Acquisition Functions and Sequential Decision-making

At iteration n , Bayesian optimization constructs an acquisition function $a_n(\mathbf{x})$ from the current posterior and proposes

$$\mathbf{x}_{n+1} \in \arg \max_{\mathbf{x} \in \mathcal{X}_{\text{safe}}} a_n(\mathbf{x}). \quad (2.6)$$

The selected point is evaluated, appended to the eligible history, and used to refit the model. Acquisition functions balance exploitation of promising means with exploration of uncertain regions, although their behavior also depends on initialization, scaling, and numerical optimization [2, 5, 12].

2.6.1 Improvement-based acquisitions

For maximization with a known incumbent value f_n^+ , the improvement of a candidate is

$$I(\mathbf{x}) = \max\{f(\mathbf{x}) - f_n^+, 0\}, \quad (2.7)$$

and expected improvement is

$$\text{EI}(\mathbf{x}) = \mathbb{E}[I(\mathbf{x}) \mid D_n]. \quad (2.8)$$

EI becomes large when improvement is probable, large in magnitude, or both [5]. With noisy observations, the best latent function value is not known exactly. Noisy expected improvement integrates over the joint posterior at previously observed baseline points and the candidate, avoiding the assumption that the largest noisy observation is the true incumbent.

Expected-improvement acquisition values can become numerically tiny over large parts of the domain. This creates flat regions and unstable numerical optimization even when the mathematical criterion is well defined. The LogEI family uses numerically stable logarithmic formulations. An exact logarithmic transformation preserves acquisition ordering, while the Monte Carlo variants also use smooth approximations to the sample-level utility with controlled approximation error. These constructions improve numerical behavior without guaranteeing exact ordering equivalence for every approximation [13].

The logarithm changes only the acquisition calculation, not the measured or reported database objective. The qLog methods used in this study are stable forms of the corresponding noisy improvement criteria.

2.7 Multi-objective Bayesian Optimization

The study considers throughput and p99 jointly. To express both as maximization objectives, define

$$\mathbf{f}(\mathbf{x}) = (f_{\text{TPS}}(\mathbf{x}), -f_{\text{p99}}(\mathbf{x})). \quad (2.9)$$

Negating p99 is a sign convention used by the optimizer; reported latency remains positive milliseconds.

2.7.1 Dominance and Pareto optimality

For two objective vectors $\mathbf{u}$ and $\mathbf{v}$, $\mathbf{u}$ dominates $\mathbf{v}$, written $\mathbf{u} \succ \mathbf{v}$, if $u_j \geq v_j$ for every objective j and $u_j > v_j$ for at least one. A configuration is Pareto optimal within an evaluated set if no other evaluated configuration dominates it. The nondominated objective vectors form the observed Pareto front.

Pareto optimality provides a partial, not total, ordering. Two configurations can both be nondominated when one has higher throughput and the other lower p99. Selecting one for deployment then requires an external preference, operational constraint, or confirmatory rule. A larger observed Pareto front is not automatically better because it may contain tightly clustered or weak trade-offs; front quality must also consider its location and coverage.

2.7.2 Random scalarization and ParEGO

Scalarization converts an objective vector into a temporary scalar objective. ParEGO uses changing weights and an augmented Tchebycheff construction to sample different trade-offs across iterations, allowing a single-objective surrogate optimization step to contribute points to a multi-objective front [14]. Random-scalarization frameworks generalize the idea and can direct sampling toward different regions of the Pareto front [15].

2.7.3 Expected hypervolume improvement

Hypervolume-based acquisition functions optimize multi-objective quality directly. Given a reference point $\mathbf{r}$ that is worse than the relevant outcomes, the hypervolume of a nondominated set P is the measure of the objective-space region dominated by P and bounded by $\mathbf{r}$:

$$\text{HV}(P; \mathbf{r}) = \lambda \left(\bigcup_{\mathbf{y} \in P} [r_1, y_1] \times [r_2, y_2] \right), \quad (2.10)$$

where λ denotes area for the two-objective case. A point that extends the front in either direction can increase hypervolume; a dominated point adds no area.

Expected hypervolume improvement (EHVI) is the posterior expectation of the additional area contributed by new candidates. qEHVI provides a differentiable Monte Carlo parallel formulation [16]. When observations are noisy, the true Pareto front is uncertain. qNEHVI integrates over that uncertainty rather than treating the observed front as exact [17]. Because hypervolume depends on the reference point and objective scale, comparative analysis must fix both before inspecting the method outcomes.

2.8 Initialization, Dimensionality, and Mixed Parameter Domains

A surrogate requires observations before it can guide search. An initial design should cover the domain without consuming an excessive fraction of the budget. Independent random samples are simple, while a scrambled Sobol sequence is a low-discrepancy design intended to fill a hyperrectangle more evenly.

The configuration space is mixed. Some knobs are continuous, while integer knobs represent counts, seconds, kilobytes, megabytes, or blocks. The optimizer operates on normalized coordinates in $[0, 1]^d$; decoding maps a coordinate back to the admissible physical value and rounds integer knobs according to the configuration-space definition. Different normalized points can decode to the same physical configuration near integer boundaries, so duplicate exclusion must occur after decoding as well as at the abstract vector level.

Dimensionality affects both coverage and model learning. With a fixed number of observations, adding knobs makes the space sparser and creates more possible interactions. General model-based configurators support categorical and mixed domains [9], while specialized high-dimensional BO methods change the surrogate or search geometry [11]. Database tuning research also shows that selecting a small set of influential knobs can substantially reduce the required measurements [18, 19]. The present framework therefore treats screening and bound construction as part of the experimental method, not as an invisible preprocessing convenience.

Some unsafe points can be rejected deterministically from configuration rules and resource bounds; unknown feasibility properties can instead be modeled by constrained BO [20]. The chosen treatment must remain separate from claims about objective-model performance.

2.9 Multi-fidelity Optimization

Multi-fidelity optimization uses cheaper approximations to a costly target evaluation. Let $z \in Z$ denote fidelity, with z^* the full-fidelity setting. The response becomes $f(\mathbf{x}, z)$, and the aim is to use low-cost evaluations at $z \neq z^*$ to improve decisions about $f(\mathbf{x}, z^*)$. General methods may treat data size, iteration count, simulation resolution, or measurement duration as discrete or continuous fidelities [21].

For database benchmarking, a shorter measurement window is a natural low fidelity. If early TPS and p99 estimates rank candidates similarly to a full run, clearly weak candidates can be stopped early. Fidelity is therefore distinct from the configuration being optimized: the same configuration is evaluated with a cheaper observation procedure. A promising short-run result is evidence for continuing measurement, rather than a substitute for the full-fidelity outcome. General multi-fidelity Bayesian optimization can model responses jointly across configuration and fidelity and choose both variables [21]. The framework instead uses a fixed prefix-and-promotion policy, whose decision flow is shown in Figure 2.2; it does not fit a joint multi-fidelity GP.

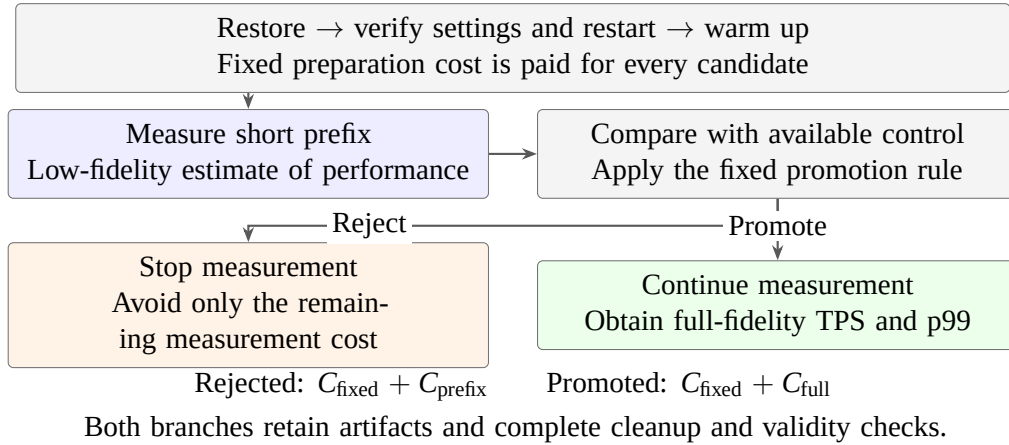


Figure 2.2: Measurement-duration fidelity through prefix-based promotion. Both branches incur the common lifecycle cost C_{fixed} , including preparation and cleanup. C_{full} includes the prefix, so promotion continues the same measurement rather than restarting it. Rejection avoids only $C_{\text{full}} - C_{\text{prefix}}$. This is a mechanism diagram, not a plot of measured savings.

Short measurements are valuable only when two conditions hold. First, their decisions must agree sufficiently with full-fidelity evidence; otherwise, promising candidates can be falsely rejected. Second, the variable measurement time saved must be large relative to fixed costs. In this framework, logical restore, restart, verification, and warm-up occur before the measured interval. A reduction in measurement duration can therefore produce a large percentage saving within the benchmark phase but only a small saving in end-to-end lifecycle time.

For a rejected fraction r under common per-candidate costs, the fractional saving relative to measuring every candidate at full fidelity is

$$S = \frac{r(C_{\text{full}} - C_{\text{prefix}})}{C_{\text{fixed}} + C_{\text{full}}}. \quad (2.11)$$

This accounting identity explains why accurate short-run rankings alone cannot establish efficiency: savings also require enough rejections and a substantial avoidable measurement cost. It assumes equal lifecycle costs and omits separate controls and failed attempts; the experimental analysis uses their recorded costs rather than treating this illustration as a runtime estimate. The other risk is a false rejection, where a weak prefix conceals strong full-run performance. Such a decision saves time but may remove a configuration that the search should have retained.

Retrospective truncation can estimate how well a prefix predicts the final result, but only a live promotion experiment measures realized stopping and wall-clock savings. Ranking fidelity, false-rejection risk, and end-to-end cost must therefore be evaluated separately.

Chapter 3

Related Work

3.1 Evolution of Automated and Self-tuning Database Systems

Research on self-tuning databases predates current machine-learning-based configuration optimizers. Early work described administration as a feedback process in which the system observes behavior, diagnoses an opportunity, applies an action, and learns from the result. Weikum et al. emphasized that this process requires both mathematical models and careful engineering [22]. Work on AutoAdmin and related systems initially focused on physical design, including indexes, materialized views, and partitions [23]. Configuration tuning differs because evaluating a server-level setting usually requires changing a running system and executing the workload rather than consulting only an optimizer cost model.

Later systems addressed mixed parameter types, interactions, and workload-dependent response surfaces. BestConfig, for example, treats the deployed system as a black box and combines space-filling samples with recursive search under a finite budget [24]. Database-specific methods can additionally use internal metrics, query features, or prior tuning histories. This produced Bayesian, reinforcement-learning, transfer, contextual, and search-space-reduction approaches. Across these families, automation can cover candidate selection alone or extend to the stateful process of applying, measuring, validating, and recovering from a recommendation.

3.2 Bayesian-optimization-based DBMS Tuning

Bayesian optimization is prominent in DBMS tuning because it targets expensive black-box observations. Systems in this family nevertheless differ in their initial evidence, workload representation, objective, and operational scope. Table 3.1 summarizes the approaches most relevant to this study.

iTuned established the expensive-response-surface view, while OtterTune and ResTune show how historical tasks can accelerate a new one. Such transfer also depends on the provenance of DBMS versions, hardware, workloads, and knob definitions. An enterprise evaluation of OtterTune found performance gains but also deployment and measurement issues that were less visible in synthetic studies [25].

CGPTuner’s distinction between an online objective, which penalizes poor intermediate configurations, and an offline objective focused on the best point directly motivates evaluation across the full candidate budget [6]. More broadly, Zhang et al. show that end-to-end results depend jointly on knob selection, configuration-space design, and the optimizer [4]. These

findings make a controlled comparison of acquisition strategies meaningful only when the surrounding space and protocol are held fixed.

Table 3.1: Selected model-based DBMS tuning systems

System	Primary mechanism	Relevant distinction
iTuned	Response-surface modeling and adaptive experimental design	Integrates candidate selection with an experiment executor and emphasizes non-monotonic parameter effects [3].
OtterTune	Knob and metric selection, workload mapping, transfer, and a Gaussian-process model	Uses a cross-workload knowledge base to reduce target evaluations [1].
ResTune	Resource modeling and meta-learning under service requirements	Combines performance, resource cost, and service-level constraints [26].
Tuneful and CGPTuner	Incremental or contextual tuning under changing conditions	Treat tuning cost and intermediate candidate quality as part of the objective [6, 27].

3.3 Reinforcement Learning and Alternative Approaches

Reinforcement learning casts tuning as interaction between an agent and an environment: the agent observes state, proposes a configuration, receives a performance reward, and updates its policy. CDBTune applies deep deterministic policy gradient to a high-dimensional continuous knob space [28]. QTune augments this formulation with SQL-query features and database state so that recommendations can respond to workload structure [29]. These methods can express rich state and sequential effects, but comparisons with BO must account for pretraining and all target-system interactions. The present study therefore uses them to establish an alternative formulation rather than attempting a cross-paradigm ranking.

Useful alternatives also exist without a learned policy or probabilistic surrogate. BestConfig performs recursive search, random sampling offers a simple undirected baseline, and scrambled Sobol sequences improve space filling without using objective feedback [24]. SMAC uses tree-based surrogates suited to mixed and categorical configuration spaces [9]. These approaches frame the baseline question for this study: whether Gaussian-process modeling produces better evaluated candidates than undirected coverage under the same budget.

3.4 Knob Selection and Search-space Reduction

Modern DBMSs expose more settings than a small experiment can explore reliably. Adding knobs makes initialization sparser, increases possible interactions, and introduces domains whose safe ranges may be uncertain. Search-space reduction therefore affects both sample efficiency and safety.

OtterTune ranks knobs from observed relationships with performance, although such rankings depend on the coverage and quality of the trials [1]. Kanellis et al. show that small influential subsets can retain much of the benefit of broader spaces, while cautioning against a universal subset independent of workload and system [18]. LlamaTune instead reshapes the search through randomized dimensionality reduction, biased sampling of special values, and

bucketization [19]. Together, these studies show that an apparent optimizer improvement may originate partly from representation and sampling choices. The present framework retains an interpretable screened eight-knob space, with its selection and bounds reported as part of the experimental method.

3.5 Positioning of the Framework and the Research Gap

Prior work establishes that automatic tuning can improve database performance in specific contexts, that transfer and workload features can accelerate adaptation, and that BO and RL can navigate interacting knobs. It also shows that recommendation quality alone does not establish trustworthy operation. State contamination, uncertain activation, failure attribution, temporal drift, and incomplete evidence can all compromise an otherwise promising optimizer. Production-oriented studies consequently motivate measuring the cost and harm of intermediate candidates as well as the final recommendation [3, 25, 27].

The framework addresses an evidential gap in assessing how tuning strategies use an expensive evaluation budget. It evaluates a throughput-oriented acquisition, random scalarization, direct noisy hypervolume improvement, and two undirected baselines under equal logical candidate budgets. The Bayesian methods begin from the same measured initialization but train on isolated subsequent histories. Throughput and p99 remain separate objectives so that candidate quality across the budget and its trade-offs remain visible rather than being hidden in a fixed scalar reward or a single best endpoint.

Every eligible observation also passes the same restoration, activation, measurement, failure, rollback, and provenance controls. This supports a fairer comparison of candidate-selection methods and preserves the evidence needed to distinguish optimizer behavior from infrastructure failure. The framework is evaluated on a controlled synthetic workload and is not presented as an autonomous production controller.

Finally, shorter benchmark prefixes are evaluated against full lifecycle cost. The analysis distinguishes retrospective ranking fidelity from live promotion, false-rejection risk, runtime-model accuracy, and realized savings. The central question is therefore whether adaptive search allocates an equal logical budget more dependably to useful regions when configuration space, system state, and evidence rules are held constant. Comparisons among acquisition functions and undirected baselines provide the evidence for that broader question; they do not require the selection of a universal winner.

Chapter 4

Framework Design and Implementation

4.1 System Requirements and Design Principles

A configuration experiment changes both server settings and database state. Without explicit control of these changes, an apparent performance difference can reflect an incomplete activation or the effects of a preceding workload. The framework therefore treats verified state as a prerequisite for measurement: each observation begins from an authenticated dataset baseline, applies only allowlisted settings within defined bounds, and checks the active values before starting the benchmark. The same checks apply regardless of how the candidate was generated, so optimization cannot relax the conditions under which its own proposals are accepted.

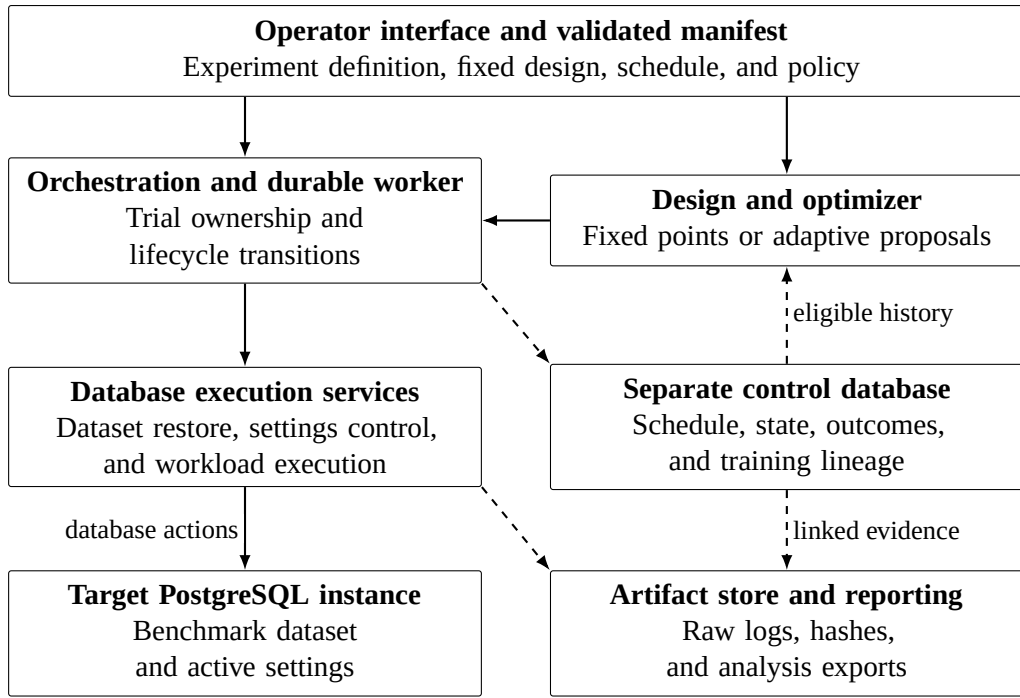
The cost and duration of database evaluation also make recovery part of the measurement design. A process interruption must not silently repeat a measured candidate, skip a budget position, or introduce a new proposal in place of a failed one. Durable records consequently distinguish the logical candidate slot, the physical observation, and its infrastructure attempts. This distinction allows one shared observation to supply several permitted optimization histories while keeping retries separate from additional search effort.

Reproducibility requires those identities to remain connected throughout the experiment. The framework preserves the specification, execution order, configuration snapshots, training lineage, raw measurements, and failure records, rather than retaining only successful objective values. Together, controlled mutation, verified measurement, durable recovery, fair budget accounting, and auditable evidence define the responsibilities of the architecture.

4.2 Overall Architecture

The architecture separates the benchmark target from the services that direct and record its evaluation. The target PostgreSQL instance holds the experimental dataset; a separate control database holds campaign schedules, trials, transitions, and results. Restarting or restoring the target therefore leaves the information needed to supervise and recover the experiment available. File-based artifacts complement the control database by retaining transaction logs and reports whose contents are too detailed to represent only as summary rows. Figure 4.1 connects these responsibilities along the path from an experiment definition to retained evidence.

An operator starts a campaign from a validated manifest that defines its configuration space, fixed candidates, schedule, and evaluation policy. Persisting this definition before execution prevents runtime decisions from silently changing the comparison. Fixed-design slots already contain their candidates, whereas an adaptive slot is materialized only when its permitted pre-



Solid arrows: execution control or proposals. Dashed arrows: state and evidence.

Figure 4.1: Architecture of the configuration-evaluation framework. Orchestration coordinates proposal generation and the durable worker; the controller and workload adapter act on the benchmark target. A separate control database preserves execution state and eligible optimization history, while the artifact store retains raw measurements and authenticated exports.

ceding observations are available. Orchestration then passes the persisted proposal to a durable worker, which owns the evaluation lifecycle and records its progress in the control database.

The worker delegates setting changes to the controller and benchmark execution to the workload adapter. This division keeps PostgreSQL-specific activation rules separate from acquisition logic while giving every optimizer the same measurement interface. The implementation uses Python with Psycopg for database access and Pydantic for configuration validation; Typer exposes operator commands, and FastAPI supplies health and reporting endpoints. PyTorch and BoTorch provide the numerical and Bayesian optimization components [30].

The control database is also the boundary between execution and analysis. Ordered, hash-checked SQL migrations maintain its schema, while constraints protect immutable schedule positions and proposals. Training lineage records which valid earlier observations support each adaptive proposal. Artifact registrations connect the corresponding files through relative paths, sizes, and SHA-256 digests, allowing reports to resolve an objective value back to its configuration and raw measurement.

4.3 Configuration-space Representation

Proposal generation operates on a normalized configuration vector in $[0, 1]^8$, while PostgreSQL accepts values in parameter-specific units and domains. An ordered codec connects these representations so that model inputs, fixed designs, and applied settings have the same meaning. The parameter order is part of the experiment definition; the selected parameters and their native bounds are specified in Table 5.3 in Chapter 5.

Integer coordinates decode into inclusive bins spanning the allowed values. Continuous coordinates use affine scaling followed by six-decimal quantization, and inverse encoding maps an integer value to its bin midpoint. Because multiple normalized points can decode to the same physical configuration, duplicate detection operates on canonical JSON representations of the decoded settings. Exclusion therefore prevents different floating-point vectors from consuming additional candidate slots for identical settings.

Decoding establishes a candidate within the research space, but it does not establish that the responding server can accept it. The controller therefore validates the decoded values independently against an allowlist and live `pg_settings` metadata, including types, units, bounds, and activation contexts. After applying a candidate through `ALTER SYSTEM`, it verifies the active settings and pending-restart state. Integer settings require exact equality; real-valued settings permit a small representation tolerance. This second validation boundary connects the optimizer’s abstract input to the actual server state used for measurement.

4.4 Optimization Components

The optimization interface returns one configuration for one logical budget position. Random and scrambled-Sobol search generate fixed designs without performance feedback, providing baselines for the value of adaptation. The Bayesian methods instead fit a response model to their eligible history and score potential evaluations through an acquisition function. Their differences are confined to proposal generation, as summarized in Table 4.1; restoration, measurement, and validity remain responsibilities of the shared execution path.

Table 4.1: Candidate-generation mechanisms

Method	Response model	Proposal criterion
Random	None	Seeded pseudorandom sampling.
Sobol	None	Scrambled low-discrepancy sampling.
qLogNEI	One Gaussian process	Log noisy expected improvement in TPS.
qLogNParEGO	Two independent Gaussian processes	Log noisy improvement under random scalarization of TPS and negative p99.
qLogNEHVI	Two independent Gaussian processes	Log noisy expected hypervolume improvement relative to a fixed reference.

4.4.1 Model Fitting and Proposal Generation

An adaptive proposal may depend only on valid observations at earlier schedule positions in the same seed and method history, together with that seed’s shared Bayesian initialization. Excluding controls, calibration observations, failed measurements, and rival adaptive histories preserves the information available to each method. The shared initial design supplies a common starting point; subsequent isolation prevents one method’s adaptive outcomes from influencing another method’s proposals.

Throughput-only optimization fits a `SingleTaskGP` to TPS. The multi-objective paths fit independent models for TPS and negative p99 and combine them in a `ModelListGP`, placing both objectives in maximization coordinates. Standardized outcomes accommodate their different numerical scales, and double-precision tensors support fitting and acquisition evaluation. The logarithmic improvement acquisitions retain the objective interpretations developed in Chapter 2 [13, 17].

Table 4.2 specifies the surrogate used by all three Bayesian methods. Its covariance is the radial basis function (RBF) kernel with automatic relevance determination (ARD),

$$k(x, x') = \exp \left[-\frac{1}{2} \sum_{j=1}^8 \frac{(x_j - x'_j)^2}{\ell_j^2} \right], \quad (4.1)$$

where each ℓ_j measures variation along one normalized parameter axis. These settings follow the defaults of the locked BoTorch 0.17.2 and GPyTorch 1.15.2 implementation [31]. Specifying the dependency version matters because the application supplies neither a custom covariance nor a custom likelihood. Each adaptive decision constructs and fits a fresh model on the complete eligible history, containing twelve observations before the first adaptive proposal and twenty-nine before the last. The resulting hyperparameter estimates condition the acquisition calculation; their uncertainty is not integrated through posterior sampling.

Table 4.2: Gaussian-process specification used in the experiments

Component	Specification
Inputs	Eight saved codec coordinates in $[0, 1]^8$; linear native-unit scaling with integer decoding and continuous quantization. No logarithmic input transformation or additional model input transform.
Outputs	Raw TPS and, for multi-objective methods, negative p99 in ms. Each output is separately centered and scaled to unit sample standard deviation using its current training history; no log-output transformation.
Mean and covariance	Learned constant mean; unit-amplitude RBF covariance, eight ARD length scales; no separately fitted output scale.
Length-scale prior	Independently $\log \ell_j \sim \mathcal{N}(\sqrt{2} + \frac{1}{2} \log 8, 3)$, with $\ell_j \geq 0.025$. The second normal parameter is variance.
Observation noise	Learned homoskedastic Gaussian noise for each output; $\log \sigma_n^2 \sim \mathcal{N}(-4, 1)$ and $\sigma_n^2 \geq 10^{-4}$ in standardized-output units. Per-observation noise variances are not supplied.
Output dependence	One <code>SingleTaskGP</code> for TPS, or two independent models combined in <code>ModelListGP</code> ; no cross-output covariance.
Fitting	Prior-regularized exact marginal log likelihood, giving maximum a posteriori hyperparameter estimates; summed objective for the model list. <code>fit_gpytorch_mll</code> uses SciPy L-BFGS-B and its default retry policy (at most five attempts per fit on retry-triggering failures).
Numerics and reuse	Double precision, deterministic proposal seed, fresh fit at every adaptive slot; no warm-started hyperparameters.

Acquisition maximization uses a finite pool to make candidate selection explicit in the mixed integer–continuous space. A deterministic scrambled-Sobol pool contains 2,048 normalized points, each scored as a single-candidate proposal using 256 Sobol quasi-Monte Carlo samples. Decoding proceeds in descending acquisition order until a physical configuration outside the exclusion set is found. Although the acquisition names retain the batch prefix q , this path evaluates $q = 1$. It therefore implements sequential search over a finite candidate pool rather than unrestricted continuous acquisition optimization.

The acquisition seed also determines qLogNParEGO’s positive scalarization weights, normalized to sum to one. qLogNEHVI instead uses the same fixed reference point as the hyper-volume analysis, with its value and calibration rationale specified in Section 5.5. Persisting the selected configuration, acquisition value, seed derivation, and training identities makes each adaptive decision traceable to its inputs.

4.5 Candidate Evaluation Lifecycle

Evaluation begins when a worker claims the trial through an expiring lease. The worker restores the approved dataset and verifies its identity before allowing a candidate to affect the target. Resource and workload checks then establish that the requested evaluation can proceed within its prescribed context. Only after these prerequisites pass does the controller apply the settings, restart PostgreSQL, and verify the active configuration. A fixed warm-up prepares the restored and restarted system for the timed workload.

The measurement phase captures PostgreSQL statistics before and after the run and retains both client output and transaction-level logs. Throughput comes from the benchmark output, while the transaction latencies supply percentile estimates. CPU, memory, I/O, WAL, checkpoint, and database counters provide context for validity checks and diagnosis. Keeping this telemetry separate from the TPS and p99 objectives allows the experiment to explain an unusual observation without silently changing what the optimizer learns.

A measurement becomes eligible evidence only after output completeness, transaction outcomes, and resource state have been checked. The worker records the result together with timings and artifact identities, then restores and verifies the pre-trial configuration. Rollback is part of the lifecycle because the setting state left by one trial must not become an implicit starting configuration for another. Its outcome remains linked to the measurement, so an apparently successful benchmark cannot conceal a failed finalization.

Each phase transition is durable, and stable application, snapshot, and measurement identities allow recovery to recognize work already performed. Atomic measurement markers distinguish a complete retained result from a partial file. This combination makes the lifecycle resumable at meaningful boundaries without assuming that a lost client process implies a lost or unexecuted database action.

4.6 Safety, Failure Handling, and Resumption

Campaign readiness authenticates the manifest, candidate design, schedule, baseline, schema, and target identity before execution begins. Per-trial gates then check restore equivalence, configuration bounds, resource state, active settings, workload completeness, rollback, and target health. Readiness checks protect the experiment as a whole, while trial checks address conditions that can change during execution. Neither layer treats poor objective values as a reason to erase an otherwise valid observation.

Failure classification follows the source of the failure because recovery can otherwise distort the search budget. A restore or fingerprint failure occurs before a valid candidate evaluation and permits a bounded retry of the same persisted proposal. A candidate, activation, workload, or measurement failure terminates its assigned slot and remains visible in the comparison. Exhausting the infrastructure retry allowance pauses the campaign rather than generating replacement candidates. Section 5.5 defines the numerical retry allowance and its statistical treatment.

An interruption during measurement requires reconciliation when durable state and partial artifacts do not establish a completed observation. The dedicated operator-assisted path operates with the campaign paused, restores and verifies the target configuration, retains the partial evidence, and returns the same candidate slot to a retry-pending state. This path complements ordinary retry handling; it does not treat an incomplete measurement as an eligible observation.

Leases maintain exclusive ownership of a trial during long-running actions. The worker renews its lease and stops acting on a confirmed loss of ownership; a replacement worker reconciles an expired lease against the durable state before continuing. Trial-specific PostgreSQL

application names identify orphaned workload sessions so that they can be terminated before resumption. Configuration snapshots similarly support rollback during both normal completion and exception handling. Any rollback failure is retained alongside the original outcome instead of replacing its cause.

4.7 Evidence Integrity, Testing, and Reproducibility

The evidence model preserves experimental roles as well as numerical values. Infrastructure validation, calibration, primary comparison, secondary analysis, and confirmation remain distinct populations, and controls remain distinct from search candidates. These identities govern both training eligibility and report construction. As a result, the analysis can reproduce logical method histories while counting shared initialization once as physical work and keeping infrastructure attempts separate from candidate allocations.

Integrity checks connect the frozen specification to the final presentation. Hashes authenticate manifests, schedules, proposals, measurements, and exported analysis inputs; atomic writes prevent incomplete artifacts from appearing as finished outputs. Reports use stable ordering and numeric formatting so that equivalent authenticated inputs yield reproducible tables and figure data. The resulting evidence chain supports verification of how a reported endpoint was obtained, including the failures and exclusions that affect its meaning.

Testing concentrates on the boundaries where a defect could alter that meaning: configuration encoding and proposal selection, database constraints and migrations, interrupted execution and recovery, and deterministic analysis and export. A result-blind rehearsal constructs synthetic ledgers with the primary campaign topology and exercises the reporting path without contacting the benchmark target. These checks validate accounting and recovery behavior; they do not substitute for workload measurements. Frozen manifests, dependency locks, pinned container identities, and retained lineage make the procedure reconstructable, while repeated performance remains subject to measurement noise. The experimental design addresses that remaining variability through controlled workload conditions, interleaved defaults, and paired seed-level analysis.

Chapter 5

Experimental Methodology

5.1 Research Design and Experimental Environment

The primary experiment compares five candidate-selection methods within a five-seed paired design. The selection method is the experimental treatment; the dataset, workload, parameter bounds, resource envelope, and evaluation lifecycle remain fixed. Pairing methods within seeds supports comparisons under common experimental conditions and preserves the dependence introduced by shared Bayesian initialization. Accordingly, the seed is the unit of analysis. Primary pairing is at the experiment-seed level; workload randomness is assigned separately to each physical observation. Successive candidates within an adaptive history and transactions within a benchmark run do not constitute independent experimental replications.

Calibration and infrastructure validation establish the conditions of this comparison without supplying its optimizer training data. Secondary analyses evaluate shortened measurement, and confirmation repeats selected configurations under a separate blocking design. Maintaining these roles prevents a useful diagnostic or calibration observation from silently enlarging a method’s search budget. All primary methods are therefore evaluated on the same evidential basis, even though the broader methodology uses several forms of validation.

The labels F2, F3, and F4 identify the measurement procedures retained in the implementation. F2 is a short prefix: 60 or 120 s retrospectively and 60 s in the live promotion procedure. F3 is the full 600-second measurement after 600 s of warm-up. F4 repeats fixed configurations in four common-seed blocks using the same F3 duration; its additional evidence comes from replication. No F1 procedure is specified or evaluated in the frozen experimental protocol. The labels are retained for traceability to the manifests and should not be read as four consecutively tested fidelity levels.

Execution was divided into two blocks under the same primary protocol: the first used seeds 88408573, 1418705027, and 642754166, and the second used the two reserved seeds 1902413987 and 740267717. Before the second block began, the continuation record froze the rule for combining all five seed-level summaries, the five endpoint definitions, and the statistical comparisons. The first block’s results informed the decision to execute the reserved seeds but did not change the benchmark profile, search space, methods, budgets, validity rules, or analysis formulas. The second block’s deterministic schedule and fixed candidate designs were generated and authenticated before measurement; its results were then authenticated and analyzed separately before the combined analysis. Pooling assigns equal weight to each of the five seeds and retains drift-flagged seeds, rather than treating individual candidates as independent replications. The retrospective shortened-measurement analysis (F2) and finalist confirmation (F4) retained their already fixed source populations from the first three seeds. These populations

were deliberately not expanded after the second block, preserving the original prefix evaluation and finalist selection rather than selecting them again from a larger observed pool.

The target is a PostgreSQL 18.1 Linux container with fixed resource limits on a Windows host. The benchmark client and control database operate outside the target container, keeping their resource allocation distinct from the server envelope under evaluation. Table 5.1 summarizes the environment and full-fidelity profile. The four-CPU allocation limits container resources; it does not reserve four dedicated physical cores.

Table 5.1: Experimental environment and full-fidelity benchmark profile

Property	Specification
Host	Windows 11, build 10.0.26200; AMD64; 16 logical CPUs; approximately 31.3 GiB physical memory
Target DBMS	PostgreSQL 18.1, Debian-based Linux image pinned by SHA-256 digest
Container limits	4 CPUs, 4 GiB memory, 1 GiB shared memory, PID limit 512
Storage	Dedicated Docker named volume on local ext4 storage
Execution environment	CPython 3.13.12; Docker Engine 29.1.2; Compose 2.40.3-desktop.1
Dataset	pgbench scale 500; 50,000,000 account rows
Workload	Built-in TPC-B-like transactions; 32 clients and four client threads
Observation profile	Logical restore, unconditional restart, 600 s warm-up, 600 s measurement
Instrumentation	Transaction logs, PostgreSQL statistics and I/O timing, client and container resource measurements

Environment validation addresses the risk that resource or storage failures invalidate an otherwise controlled run. The target uses data checksums and a container init process, while readiness requires fifteen consecutive logical restores passing both fingerprint tiers. Runtime artifacts reside outside synchronized directories, with at least 50 GiB free capacity at readiness. The image digest, dependency lock, baseline identity, and hashed manifests bind these conditions to the reproduction record. They fix the intended environment without assuming that host interference or performance drift disappears.

The implementation, automated tests, experiment manifests, and supporting documentation are maintained in the thesis software repository at <https://github.com/asuta2/CHARM-DB> [32]. The repository connects the experimental procedure to its executable code and locked dependency specification. Appendix B identifies the relevant entry points and explains how the separately stored measurement artifacts support evidence verification and report reproduction.

5.2 Dataset Preparation and Candidate-level Restoration

The workload updates balances and appends transaction history, making the database state itself a potential confounder. Restoring only between methods would leave later candidates dependent on the workload history of their own search. Every candidate and default control therefore receives a separate logical restoration of the same canonical dataset. At scale 500, this dataset contains 50,000,000 accounts, 500 branches, 5,000 tellers, and an initially empty history relation.

Canonical preparation performs `VACUUM (FREEZE, ANALYZE)` and a checkpoint before snapshot approval. Each logical restoration reloads that snapshot, performs `VACUUM`

(ANALYZE), and checkpoints the restored database before configuration activation. Both warm-up and measurement use `pgbench`'s `-no-vacuum` option, keeping its automatic preliminary maintenance outside the evaluation procedure. PostgreSQL autovacuum remains enabled; the option controls client-initiated preparation, not the server's background maintenance.

Restoration is accepted under an explicit equivalence contract. The exact fingerprint checks canonical schema and content evidence, relation counts, settings, PostgreSQL version, and container image identity. A second tier checks physical properties, including database and relation sizes, tuple estimates, maintenance counters, and the WAL flush boundary. Tolerances account for physical metadata that need not reproduce bit for bit: size differences, for example, must remain within the larger of 1 MiB and 2% of the reference value. This distinction makes the controlled starting condition testable without equating a logical restore with an identical physical image.

The choice of logical restoration balances this equivalence requirement against evaluation cost. A three-repetition comparison at each of scales 100 and 500 contrasts logical reload with stopped-volume physical restoration. At scale 500, both satisfy the fingerprints, with median restoration times of 197.834 s and 241.372 s, respectively. This bounded validation supports the logical path; it does not provide an upper-tail runtime estimate. Lifecycle analysis therefore uses each observation's actual restoration time. Copy-on-write restoration is outside the comparison because the target volume lacks a verified atomic clone primitive.

After restoration, every observation applies its requested settings and restarts PostgreSQL unconditionally. The common restart removes differences in activation procedure between parameters that require a restart and those that permit reload. It does not flush all host or virtualization-layer caches, and restoration itself accesses storage. The measurement condition is consequently a verified restored database followed by a fixed warm-up, rather than an assumed cold-cache state.

5.3 Workload, Warm-up, and Duration Calibration

The workload is `pgbench`'s built-in TPC-B-like transaction script, which updates account, teller, and branch balances and inserts a history row. It supplies a controlled synthetic OLTP workload rather than a certified TPC-B benchmark [7]. With fixed concurrency and no imposed arrival-rate limit, clients issue another transaction after the preceding one finishes. Throughput and p99 therefore characterize this closed-loop operating condition; they do not describe latency under an independently fixed external request rate.

Workload calibration seeks an operating region in which configuration changes produce meaningful server-side trade-offs without making the client the bottleneck. The saturation design crosses scales 100, 250, and 500 with client concurrencies 4, 8, 16, 32, and 64 under default, throughput-leaning, and latency-leaning anchors. Its 45 probes use 60 s of warm-up and 120 s of measurement, with a deterministic reseed before each scale–anchor ladder. This limited restoration scope is suitable for locating an operating region; the probe measurements remain separate from the restored primary comparison.

The saturation criterion combines throughput growth of at most 10% with p99 growth of at least 20% as concurrency increases. Client CPU use is checked against the four-thread capacity so that client limitation does not masquerade as server saturation. These checks support scale 500 and 32 clients. Trade-off validation at this operating point uses eight fixed configurations interpolated between the anchors, with individual restores, 120 s of warm-up, and 600 s of measurement. Requiring at least three nondominated configurations ensures that the workload

exposes variation relevant to a two-objective comparison, rather than merely assuming that throughput and latency provide distinct selection pressures.

Warm-up validation addresses startup effects within that operating region. Four representative configurations are monitored for 1,200 s from restored and restarted states, using 60-second windows of performance and resource activity. Cache-memory and throughput transitions extending into the 480–600-second interval motivate a common 600-second warm-up. Checkpoint-related variation is treated differently: because it recurs during workload execution, removing a startup interval does not remove it from the response being measured.

Choosing a measurement interval requires agreement in objective values and configuration rankings, beyond the startup behavior addressed by warm-up. Duration validation compares 300-second prefixes and standalone runs with 600-second observations across six representative configurations and three randomized blocks. Acceptance of the shorter duration requires median TPS disagreement within 5%, p99 disagreement within 1 ms, rank correlation of at least 0.90, and Pareto-membership agreement of at least 0.80, together with standalone-default agreement. The standalone p99 criterion rules out the 300-second alternative: disagreements of 1.806 and 7.745 ms make a three-value median of at most 1 ms impossible even if the third difference is zero. The methodology therefore retains the conservative 600-second duration under the validation’s futility rule, without treating every equivalence criterion as demonstrated. A longer measurement is not a stationarity guarantee; in particular, some checkpoint intervals exceed the measurement window.

The resulting full-fidelity observation, denoted F3, separates 600 s of warm-up from 600 s of measurement. Warm-up transactions do not enter the objectives. Full-run TPS is parsed from the benchmark output, while p99 is computed from successful transaction latencies pooled across client logs. After conversion from microseconds to milliseconds, linear interpolation at zero-based rank $0.99(N - 1)$ among N sorted latencies defines the percentile. Pooling samples before computing p99 avoids assigning equal weight to thread percentiles with different sample counts. Any transaction failure prevents the run from supplying valid primary objectives.

5.4 Default Controls and Search-space Finalization

The default reference is the target’s reset configuration under the fixed instrumentation and resource envelope. It is a reproducible comparator for this experiment rather than a universal PostgreSQL default. An initial block of five observations checks zero-failure execution, precision, and drift. The two-sided 95% Student- t interval half-width must be within 5% of mean TPS and 5 ms for p99, and fitted first-to-last changes must satisfy the same limits. This gate establishes acceptable launch conditions; later controls remain necessary to reveal temporal variation during the longer comparison.

Knob screening reduces an eleven-parameter pool to a tractable space for the fixed search budget. It uses 32 joint scrambled-Sobol configurations and default controls at physical positions 1, 18, and 35. For each knob, the screening score is the larger absolute partial rank correlation with TPS or p99 after controlling for the other ten ranked knobs. The registered rule retains safe knobs with a score of at least 0.20, requires `shared_buffers`, and sets a minimum of eight dimensions. Durability-reducing settings are not considered. The measured evidence and resulting decision are summarized in Table 5.2.

The failed drift gate is not reclassified as a pass. A documented post-result amendment permits the completed screen to guide the next stage as exploratory, drift-limited evidence, and the planned one-at-a-time follow-up checks were not performed. The final space is therefore a transparent engineering choice, not an unbiased or empirically optimal ranking of PostgreSQL

Table 5.2: Knob-screening evidence and final decision

Decision step	Evidence and outcome
Completed screen	32/32 Sobol configurations and 3/3 default controls produced valid, zero-failure measurements.
Threshold retention	max_wal_size (0.902), shared_buffers (0.361), effective_cache_size (0.265), random_page_cost (0.252), checkpoint_completion_target (0.250), work_mem (0.249), and max_parallel_workers_per_gather (0.215).
Minimum-size addition	checkpoint_timeout (0.176), the highest-scoring remaining safe knob, was added to reach eight dimensions.
Excluded	maintenance_work_mem (0.128), effective_io_concurrency (0.127), and default_statistics_target (0.073).
Validity qualification	Fitted p99 drift was 6.438 ms against the registered 5 ms limit; the original screen therefore did not pass this gate.

knobs. It nevertheless fixes the same nontrivial search problem for every method before the primary comparison. Table 5.3 reports that space in codec order together with the values used by the default controls.

Table 5.3: Primary configuration space and default control values in native units

Parameter	Lower	Upper	Default	Unit
shared_buffers	16,384	183,500	16,384	8 kB blocks
effective_cache_size	131,072	524,288	524,288	8 kB blocks
work_mem	1,024	32,768	4,096	kB
checkpoint_timeout	300	1,800	300	seconds
checkpoint_completion_target	0.70	0.95	0.90	dimensionless
max_wal_size	512	4,096	1,024	MB
random_page_cost	1.0	4.0	4.0	dimensionless
max_parallel_workers_per_gather	0	4	2	workers

The upper bound for `shared_buffers`, 183,500 blocks, is approximately 1.4 GiB or 35% of the target’s 4 GiB memory limit. It is an operational cap in the controller’s allowlist, leaving memory for backend operations and the operating-system cache. This allocation is consistent with PostgreSQL’s guidance to start near 25% on a dedicated server and its observation that allocations above 40% are unlikely to improve performance [7]. The exact 35% cutoff is an engineering boundary; calibration supplies no comparison establishing it as an optimal cutoff. All methods use this same bounded domain, and the selected configuration’s proximity to its edge is examined in Section 6.4.3.

All parameters have integer domains except `checkpoint_completion_target` and `random_page_cost`, which use continuous domains with the quantization defined in Chapter 4. Bounds and hard execution gates provide the common validity policy. A learned feasibility model is not fitted because calibration provides no defensible feasible–infeasible candidate boundary: the screening observations have no benchmark failures, and the sole memory-headroom value below 20% belongs to a default control. Latency therefore remains an objective rather than a learned feasibility label.

5.5 Primary Fixed-Budget Experiment and Analysis

5.5.1 Budget Allocation and Shared Bayesian Initialization

Random and scrambled-Sobol search establish the quality attainable without performance feedback. Throughput-only qLogNEI tests adaptation toward a single objective, while qLogN-ParEGO and qLogNEHVI test scalarization-based and hypervolume-based treatment of the throughput–latency trade-off [13, 14, 17]. The models and proposal mechanisms are those of Chapter 4; the experimental comparison changes selection while holding measurement conditions constant. Both TPS and p99 are evaluated for every method, so qLogNEI’s latency and hypervolume endpoints characterize the trade-offs of throughput-directed search.

Each method receives thirty logical candidate slots per seed. The three Bayesian methods share twelve physically measured scrambled-Sobol initialization points and then receive eighteen adaptive proposals each. Sharing makes their comparison conditional on identical initial information and avoids measuring that information three times. Random and Sobol instead receive thirty separately generated points each. The Bayesian histories remain isolated after initialization, with only valid preceding observations admitted as feedback. Controls characterize measurement conditions and never train a surrogate.

5.5.2 Candidate Budgets, Seeds, and Temporal Interleaving

The fixed seeds are 88408573, 1418705027, 642754166, 1902413987, and 740267717, derived from registered SHA-256 labels independently of performance outcomes. Deterministic derivations also govern design, acquisition, and schedule randomness. Each physical observation receives a workload seed derived from its primary seed and within-seed schedule position; a retry reuses that assigned seed. This arrangement separates reproducible random choices from uncontrolled variation in the measured system.

Logical budget equality does not imply equal independent physical execution costs because Bayesian initialization is shared. Counting that measurement work once gives 126 candidate observations and five controls per seed, as shown in Table 5.4. Infrastructure attempts can add runtime without enlarging this allocation. Equal candidate budgets also do not imply equal optimizer computation time.

Table 5.4: Logical budgets and physical observation allocation

Allocation	Logical slots	Physical observations
Random per seed	30	30
Sobol per seed	30	30
Shared Bayesian initialization per seed	3×12	12
Adaptive Bayesian evaluation per seed	3×18	54
Default controls per seed	0	5
Total per seed	150	131
Five-seed design	750	655

Methods are interleaved within each seed to distribute them across temporal conditions. Rounds 1–12 contain one random point, one Sobol point, and one shared initialization point; rounds 13–30 contain one point from each of the five methods. A deterministic permutation sets the order within each round, and controls occupy physical positions 1, 34, 66, 99, and 131. Successive seed blocks preserve this structure. Figure 5.1 relates the allocation to the common observation profile and separates permitted optimizer feedback from control-based analysis.

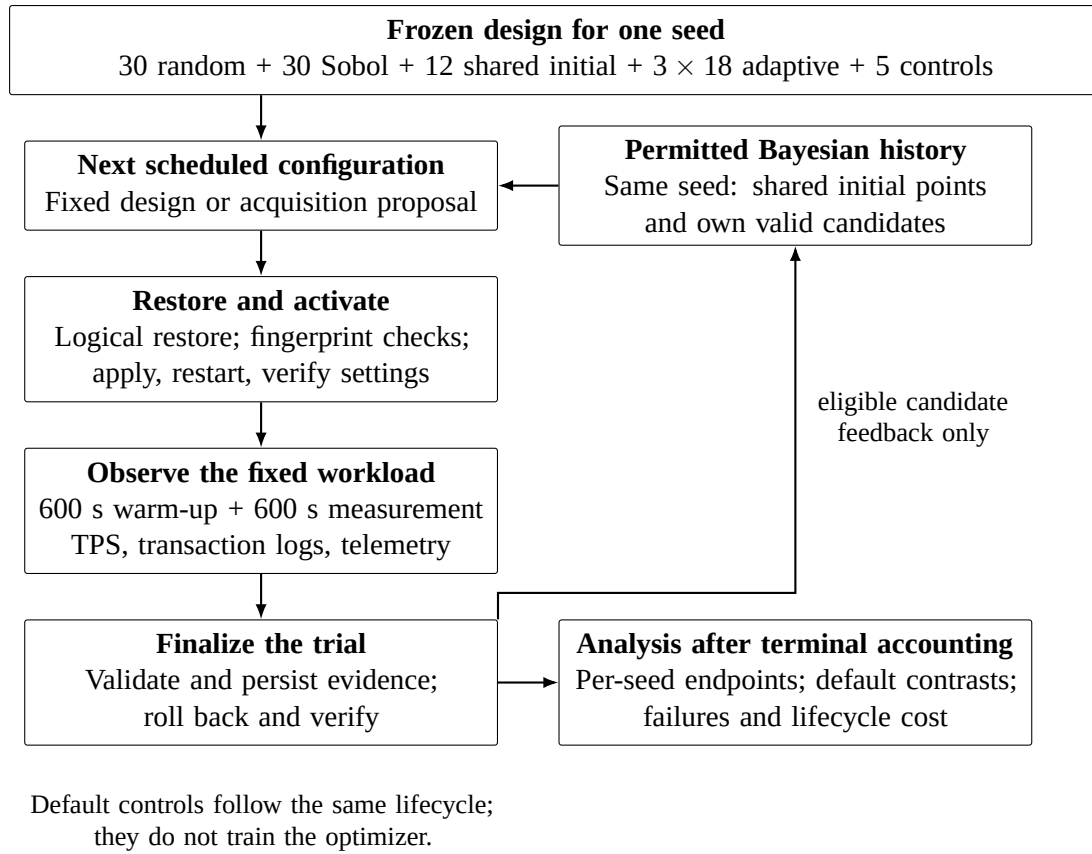


Figure 5.1: Primary evaluation within one seed. The same restored measurement profile applies to all 131 allocated observations. Shared initialization supplies three logical Bayesian histories from one physical measurement; subsequent feedback is restricted to each method’s own eligible candidates. Controls support analysis only.

5.5.3 Objectives, Reference Point, and Hard Gates

For a valid observation, let T denote TPS and L denote p99 in milliseconds. Multi-objective acquisition and hypervolume use maximization coordinates $(T, -L)$, following Chapter 2. Their fixed reference point is $(0, -40)$: zero supplies a physical lower bound on throughput, and 40 ms is the next 5 ms ceiling above 1.10 times the calibration maximum p99 of 35.83768 ms. Deriving this reference from calibration fixes the quality measure independently of the primary frontiers.

The reference defines hypervolume, not an acceptance threshold. An observation with p99 above 40 ms remains valid if it passes the common gates, although it contributes no positively dominated area above the reference. Latency remains an objective without a service-level threshold. Validity requires the manifest, target, resources, restore, active settings, benchmark output, rollback, and health to meet the execution contract. This separation ensures that poor performance is retained as a search outcome rather than reclassified as invalid evidence.

5.5.4 Endpoints and Control-relative Analysis

Five endpoints describe each method and seed at logical slot 30. Mean control-relative TPS and mean control-relative p99 characterize the quality of candidates throughout the budget. Hypervolume over valid candidates, best observed TPS, and minimum observed p99 provide complementary frontier and best-found views. This ordering reflects the primary concern with budget allocation rather than only the most favorable endpoint. Best TPS and minimum p99 can belong

to different configurations, so they are reported as separate extremes rather than combined into an unattained configuration.

Contemporaneous controls provide a local reference for the two mean-quality endpoints. If controls at physical positions a and b bracket a candidate at position p in seed s , the interpolated default for metric $M \in \{T, L\}$ is

$$\hat{M}_s(p) = M_s(a) + \frac{p-a}{b-a}(M_s(b) - M_s(a)). \quad (5.1)$$

The endpoint controls bracket the candidate sequence. If valid controls do not bracket a position, the nearest valid control supplies the reference and the missing-control condition remains visible. Interpolation uses physical schedule position, so it represents gradual change across the sequence without assuming that every observation takes equal wall-clock time.

For the valid logical candidate set V_{ms} of method m in seed s , define

$$R_{ms}^T = \frac{1}{|V_{ms}|} \sum_{i \in V_{ms}} \left(\frac{T_i}{\hat{T}_s(p_i)} - 1 \right), \quad (5.2)$$

$$D_{ms}^L = \frac{1}{|V_{ms}|} \sum_{i \in V_{ms}} \left(L_i - \hat{L}_s(p_i) \right). \quad (5.3)$$

Positive R_{ms}^T indicates higher throughput, while negative D_{ms}^L indicates lower p99; reports express the former as a percentage and the latter in milliseconds. Shared initialization enters each Bayesian logical history at its original physical position. Because the denominator includes only valid observations, these means are interpreted alongside failure counts rather than assigning artificial performance values to failed candidates.

A least-squares line through the valid controls quantifies fitted first-to-last drift within each seed. Absolute TPS change above 5% of mean control TPS or p99 change above 5 ms raises a reporting flag, without deleting observations or changing the primary stopping rule. Raw endpoints remain available beside adjusted contrasts because interpolation cannot identify abrupt, candidate-dependent, or elapsed-time-specific changes. Logical-slot trajectories, candidate distributions, and counts of observations with both strictly higher TPS and strictly lower p99 than the local default provide descriptive context. Pooled scatter plots locate configurations; method comparisons continue to use seed-level endpoints.

5.5.5 Retry, Failure, Stopping, and Statistical Rules

Recovery preserves the assigned candidate and budget position. Dataset-restore and baseline-fingerprint failures permit at most three one-attempt trials for the same proposal, supplying neither additional search slots nor training values. Exhaustion pauses the campaign with its ledger intact. Candidate, activation, workload, and measurement failures remain terminal in their allocated positions, while interrupted actions are reconciled through the durable mechanisms of Chapter 4. One interrupted primary measurement required operator-assisted reconciliation: partial evidence was retained, the target was restored and verified, and the same candidate and workload seed were retried. These rules prevent infrastructure recovery from becoming an unrecorded extension of the search.

The scientific stopping condition is the fixed candidate budget rather than an apparent plateau or favorable ranking. Terminal analysis authenticates the scheduled ledger and accounts for invalid observations and failures before forming endpoints. Primary cost reporting sums elapsed intervals of successful physical attempts and identifies restoration and fixed workload

windows within that total. Failed-attempt intervals, gaps between attempts, and proposal computation outside those intervals are excluded, although failure records remain available for audit. Secondary analyses use their separately specified lifecycle and counterfactual accounting. The timed 600-second workload is only one component of evaluation cost, which matters when assessing whether a cheaper measurement policy reduces total effort.

Statistical summaries retain each seed value, mean, median, sample standard deviation, and median absolute deviation. Descriptive 95% bootstrap intervals for means use 10,000 resamples of whole seeds with fixed analysis seeds, following the resampling principle of Efron [33]. Pairwise contrasts preserve seed pairing and report differences in natural endpoint units, together with the mean difference divided by its sample standard deviation when nonzero. Cliff’s delta supplies a supplementary distributional effect size across seed summaries [34]; it does not replace the paired comparison or make candidate rows independent replicates.

The exact two-sided paired sign-flip test compares the observed absolute mean difference with all 2^n sign assignments to the n seed differences. Its interpretation requires sign exchangeability under the null. Holm adjustment controls multiplicity across the ten method pairs separately within each of the five endpoint families [35]. With five seeds, the smallest attainable unadjusted two-sided probability is

$$p_{\min} = \frac{2}{2^5} = 0.0625. \quad (5.4)$$

Even complete directional agreement therefore cannot yield significance at 0.05 under this test. The five-seed design allocates a finite execution budget of approximately 300 h to comparing complete thirty-candidate search histories under verified restoration. Its successful attempts ultimately required 303.795 h, or about 60.8 h per seed. This budget choice supports a deliberately descriptive assessment of candidate quality, effect magnitude, and consistency; it is not a design capable of confirming superiority at $\alpha = 0.05$ with the registered two-sided test. The first-block-informed continuation further motivates retaining that descriptive interpretation. P-values and whole-seed bootstrap intervals provide supplementary context for the observed effects.

A future confirmatory study needs at least six paired seeds for an unadjusted two-sided test to attain 0.05, since $2/2^6 = 0.03125$. Retaining all ten pairwise contrasts requires at least nine seeds even to cross Holm’s first threshold of 0.05/10, since $2/2^9 = 0.00390625$. These are resolution minima, not sufficient-power guarantees. A practical planning range is 12–20 independent paired seeds, with the final count selected prospectively by simulation for a prespecified meaningful endpoint difference, 80% or 90% power, and the intended multiplicity family. At the observed cost this range requires roughly 729–1,215 h for primary measurement alone. A smaller family of scientifically targeted contrasts could reduce that requirement, provided it is fixed before collecting the confirmatory observations.

5.6 Multi-fidelity Evaluation

Shortened measurement is useful only if it identifies weak candidates early enough to save time without discarding configurations worth measuring fully. The secondary evaluation separates these questions through retrospective prefix analysis and a live promotion procedure. It tests a measurement policy, rather than fitting the joint multi-fidelity surrogate introduced in Chapter 2 [21]. The F2 and F3 measurement labels follow Section 5.1.

5.6.1 Retrospective Prefix Evaluation

The retrospective design fixes its source population to the first three primary seeds: 378 unique physical candidate observations and fifteen controls, with shared initialization counted once. Reconstructing 60- and 120-second windows from authenticated transaction logs permits comparison with full-run objectives on the same configurations. Prefix TPS is the number of successful transactions completing within a window divided by its duration; same-duration default windows provide the interpolated local reference.

Promotion requires a complete valid prefix, zero transaction failures, a source observation passing the hard safety gates, and TPS of at least 80% of the local default. No latency threshold is imposed. A missing or invalid prefix is rejected and can count as a false rejection if its full-fidelity candidate is good. Retrospective access to full-observation validity and bracketing controls provides an auditable comparison, but includes information unavailable at an online prefix decision.

Within each seed, Spearman correlation compares prefix and full-F3 rankings by control-relative throughput. A good full-fidelity candidate belongs to the top quartile under that same full-F3 criterion; rejecting its prefix constitutes a false rejection. The implemented analyzer requires correlation of at least 0.80 and zero top-quartile false rejections in every seed at both prefix lengths, together with lower pooled modeled lifecycle time at each length. This cost gate differs from the manifest’s stronger wording requiring every criterion to pass within every seed. At 120 s, two seeds promoted all their candidates and saved no time, so that stronger per-seed cost condition was not satisfied. The reported acceptance decision therefore refers to the implemented rule. Threshold sensitivity from 0.70 to 1.00 is descriptive. Because promise is defined through throughput, these checks do not establish preservation of every latency extreme or Pareto-optimal setting.

The cost model includes the preparation work already paid before a prefix can be assessed. Let C_i denote fixed restoration, activation, warm-up, verification, and finalization cost for candidate i , and let τ be the prefix duration. Then

$$C_i^{\text{prefix}} = C_i + \begin{cases} 600, & \text{if promoted,} \\ \tau, & \text{if rejected.} \end{cases} \quad (5.5)$$

A rejection saves $600 - \tau$ seconds; promotion extends the prefix to the full duration without charging another complete F3 run. Including controls and fixed lifecycle costs in the comparison prevents a reduction in timed measurement from being mistaken for the same proportional reduction in total cost. Overlap between prefix and full-run samples also limits their use as independent evidence of ranking fidelity.

5.6.2 Live Promotion Demonstration

The live design tests whether the policy can operate with information available at the decision point. It uses thirty fixed Sobol candidates with seed 748909656 and five 60-second controls at physical positions 1, 8, 15, 22, and 29. Six candidates follow each control, and each promotion decision uses only the most recent preceding default. Every physical slot retains logical restoration, unconditional restart, and 600 s of warm-up, isolating the change in measurement duration from the preparation procedure.

A valid candidate reaching the 0.80 throughput ratio after 60 measurement seconds receives a further 540 s on the same database state. The continuation starts a new client process with a deterministically derived seed, without another restore, restart, or warm-up. Combining transaction logs yields the promoted p99, while successful transaction counts across both measurement

intervals divided by 600 s yield throughput. The reconnect gap is excluded from this objective duration but retained in lifecycle cost. The resulting observation therefore represents two measurement stages rather than an uninterrupted client run.

Operational acceptance requires lower measured lifecycle time than a matched all-F3 counterfactual, runtime-model absolute relative error within 10%, and retention of failures and retries. The saving compares total lifecycle time, including retained failed infrastructure attempts, with a counterfactual that retains the same failed-attempt time, controls, and fixed costs. Runtime-model error separately compares successful lifecycle time with its predicted value. The counterfactual is thus an accounting comparator rather than an independent benchmark campaign. Rejected candidates lack full-F3 outcomes, so live operation can validate the decision mechanism and runtime model but cannot identify their false-rejection status. Its observations consequently remain secondary evidence rather than additional primary search candidates.

5.7 Confirmatory Evaluation Protocol

Confirmation addresses the variability of selected configurations separately from the effectiveness of the methods that propose them. The F4 protocol repeats four fixed finalists chosen from the pooled nondominated configurations of the first three primary seeds. Selection uses that source population to identify candidates, while the confirmation rules are fixed independently of their repeated outcomes. The pooled frontier thus defines a configuration selection pool rather than a method-level statistical sample.

The four selection roles cover throughput, latency, and frontier quality. Treatment T is the highest-throughput qLogNEI point on the pooled front. L and H are the minimum-p99 and maximum-TPS extremes of the pooled front, respectively, while E is the qLogNEHVI front point with the largest individual frozen-reference hypervolume contribution. Exact settings and source trial identities bind each label to an immutable configuration. These roles deliberately target useful frontier regions, so F4 is not a balanced repeat of all five search methods.

Each of four common-seed blocks contains the finalists and a central default control. A seeded Latin rotation places every finalist once in each non-control position, distributing its comparisons around the reference measurement. Table 5.5 specifies the resulting order. Every observation uses the same restored, restarted 600-second warm-up and 600-second measurement, without prefix rejection. F4 changes replication and control structure rather than increasing full-fidelity duration.

Table 5.5: Common-seed blocks and treatment order for confirmation

Block	Workload seed	Execution order
1	1076286005	T, E, default, L, H
2	1762147992	E, L, default, H, T
3	398860006	L, H, default, T, E
4	2004491857	H, T, default, E, L

The restored common-seed block is the unit of analysis. Raw TPS and p99 and same-block default contrasts are summarized through means, medians, standard deviations, median absolute deviations, and descriptive whole-block bootstrap intervals. All required repetitions must pass restore fingerprints and produce valid zero-failure measurements; missing or invalid evidence prevents a confirmed selection. The three-attempt infrastructure rule preserves the candidate and seed, while terminal candidate failures remain visible.

Selection requires mean same-block control-relative TPS of at least -5% , mean p99 difference of at most -1 ms, and confirmed mean TPS of at least 95% of the highest finalist mean. These gates require a practically relevant latency reduction while limiting throughput loss relative to both the default and the strongest finalist. Among eligible configurations, the lowest confirmed mean p99 determines the selection, with higher mean TPS and then the lexicographically smaller source trial identifier breaking exact ties. If no finalist qualifies, the default is retained. The rule establishes a practical configuration choice under four blocks; it does not establish general statistical superiority of an acquisition family.

Chapter 6

Experimental Results

6.1 Calibration and Validation Results

Infrastructure validation established a repeatable starting condition for configuration evaluation. The fifteen consecutive readiness restores passed both fingerprint tiers, and the restoration comparison supported the logical mechanism selected in Section 5.2. The primary observations retain their own restoration timings and verification outcomes to assess the continued validity and cost of state control.

Workload calibration supported scale 500 with 32 clients as the common operating point. All eight restored trade-off validation configurations completed, and five were nondominated, exceeding the required minimum of three. The resulting throughput–latency variation supplied an empirical basis for evaluating frontier quality alongside throughput alone. The calibration decisions in Section 5.3 established 600 s of warm-up and 600 s of measurement. The duration pilot retained this profile after its registered futility stop.

The five-observation default-reference block passed its precision and drift gates. Mean throughput was 2,722.6 TPS with a 95% interval half-width of 79.7 TPS, or 2.93%, while mean p99 was 29.927 ms with a half-width of 1.194 ms. These values characterize the calibration block rather than provide a fixed reference for the longer primary experiment. Screening subsequently produced valid measurements for all 32 candidate configurations and three controls, but its fitted p99 change of 6.438 ms exceeded the 5 ms drift limit. As documented in Section 5.4, the eight-knob space therefore rests on an exploratory, drift-limited screen accepted through a documented amendment; the one-at-a-time follow-up checks were not performed. The primary comparison consequently uses the common hard safety gates and an exploratory eight-knob space, as recorded in Table 6.1.

The departures from the original procedure and the amendments governing their interpretation are collected in Table 6.1. The table also distinguishes planned stopping and reporting flags from actual exceptions, so an incomplete calibration comparison or retained drift flag is not silently treated as a passed validation gate.

Table 6.1: Deviations from protocol, amendments, and related audit qualifications. Original evidence and registered primary endpoints are retained.

Stage	Original condition and actual treatment	Consequence
Screening recovery	Position 34 exhausted restore retries. A snapshot-backed target rebuild and three infrastructure-only validation restores preceded linked recovery of original positions 34–35.	Original positions 1–33 and their chronology gap remain in the combined screening record.
Screening interpretation	The 5 ms p99 drift gate failed at 6.438 ms. A post-result amendment accepted exploratory use, waived the one-at-a-time checks, and retired the unexecuted temporal-stability block.	The original screen remains BLOCKED_DRIFT ; the eight-knob space has exploratory support, not a passed screening gate.
Primary interruption	The ordinary retry rule covers restore and fingerprint failures. One interrupted measurement required supervised reconciliation, rollback verification, and retry of the same candidate and workload seed.	Partial evidence is retained; the retry supplies one completed observation and no additional candidate slot.
Continuation and reporting	The first three seeds informed use of the two reserved seeds. Pooling rules were frozen before continuation; a later reporting amendment presented one five-seed cohort instead of separate batch sections.	Both blocks were separately authenticated; formulas, seed weights, and endpoints are unchanged. F2 and F4 retain their original source populations.
Retrospective cost gate	The manifest requires every criterion within every seed; the analyzer checks cost reduction after pooling. At 120 s, two seeds save zero time.	Acceptance is under the implemented pooled rule. The stronger per-seed cost condition fails.
Safety reporting	The frozen report mislabels attempts as retries and sums shared logical failure attributions. The closeout erratum corrects the physical counts.	657 attempts, two failed attempts, and two retried slots; performance estimates and F4 selection are unchanged.
Planned rules retained	The duration pilot stopped for registered futility; two primary seeds exceeded transparency drift thresholds and remained included.	These are protocol-compliant stopping and retention outcomes, not additional deviations or evidence that every gate passed.

6.2 Execution Integrity and Budget Accounting

The primary experiment completed all 655 scheduled physical observations: 630 candidate measurements and 25 default controls. Every method received thirty valid logical observations in each of five seeds, giving 150 logical observations per method and 750 across the comparison. The difference between logical and physical counts arises entirely from the twelve shared Bayesian initialization observations per seed. These sixty physical observations occupy 180 logical positions across the three Bayesian histories; the remaining 270 Bayesian candidate observations belong to their separate adaptive phases. Table 6.2 reconciles the physical populations with their infrastructure attempts, keeping shared initialization distinct from method-specific acquisition.

Table 6.2: Primary physical observations and infrastructure attempts across five seeds. Each Bayesian logical history additionally includes the shared initialization; its retry attributions are not additional physical events.

Physical population	Completed slots	Attempts	Retried slots
Random	150	150	0
Sobol	150	150	0
Shared Bayesian initialization	60	62	2
qLogNEI adaptive candidates	90	90	0
qLogNParEGO adaptive candidates	90	90	0
qLogNEHVI adaptive candidates	90	90	0
Default controls	25	25	0
Total	655	657	2

Completion required 657 attempts, including two retained infrastructure failures. One attempt failed during dataset restoration and the other was interrupted during measurement. Each affected a shared initialization slot and completed on its second attempt with the same candidate and workload seed. The interrupted measurement required operator-assisted reconciliation of the partial evidence and verification of the restored target before retry. The failed attempts consumed no additional candidate budget and supplied no surrogate training observation. Because each Bayesian history includes the affected initialization points, each inherits two logical retry attributions; summing those attributions would incorrectly turn two physical failures into six. There were no candidate-failed slots, completed-but-invalid observations, or exhausted infrastructure retries. All 655 successful observations passed both restoration fingerprint tiers and supplied valid zero-failure benchmark measurements, so no terminal candidate was removed from the performance analysis.

The controlled lifecycle nevertheless imposed substantial cost. Table 6.3 reports the sum of elapsed intervals for the 655 completed physical attempts together with restoration and fixed workload windows. The completed-attempt intervals total 303.795 h, of which the successful logical restores account for 80.995 h, or approximately 26.7%. Mean restoration time was 445.161 s, substantially above the small calibration comparison’s median and therefore a more representative description of the cost actually paid in this experiment. Warm-up and measurement each account for 109.167 h of fixed windows. The remaining elapsed time includes the other lifecycle operations; these quantities do not measure active processor time or the calendar duration of the experimental program.

The distinction between successful runtime and interrupted elapsed time is necessary for interpreting recovery. An attempt left open during an interruption includes time when no benchmark was running, so its timestamp difference cannot be treated as active evaluation cost. Its

Table 6.3: Time accounting for the 655 successful physical primary attempts. Restoration, warm-up, and measurement are components of the elapsed total, not additional totals. Failed-attempt intervals and gaps between attempts are excluded.

Quantity (655 successful physical attempts)	Hours
Sum of completed-attempt elapsed intervals	303.795
Successful logical restores	80.995
Fixed warm-up windows	109.167
Fixed measurement windows	109.167

failure record and partial artifacts remain part of the evidence even though that interval is excluded from Table 6.3. The terminal audit reconciled observation identities and retained artifacts, and the offline closeout reproduced the final report byte for byte. Successful accounting and reproduction establish traceability of these results; the two drift flags examined below remain separate evidence about temporal measurement conditions.

6.3 Fixed-budget Search Effectiveness

6.3.1 Candidate Quality Across the Full Budget

The principal distinction between methods concerns the quality of candidates throughout the budget. Table 6.4 reports the registered control-relative endpoints, using the interpolated defaults defined in Equations 5.1–5.3. Across five seeds, Random and Sobol averaged throughput differences of -8.322% and -7.413% , respectively, together with p99 increases of 1.304 and 1.364 ms. The Bayesian methods remained much closer to default throughput, with mean differences between -0.946% and -0.267% , while reducing p99 by 1.232–1.530 ms. These full-history averages include initialization and show that Bayesian search allocates the budget to more favorable candidates.

Table 6.4: Candidate quality across five seeds. R^T and D^L are means of the five seed-level full-budget endpoints. Joint improvements count logical candidates dominating their interpolated default on both objectives; the 150 candidates per method are not independent replications.

Method	Joint improvements	Share	R^T (%)	D^L (ms)
Random	20/150	13.3%	-8.322	+1.304
Sobol	28/150	18.7%	-7.413	+1.364
qLogNEI	99/150	66.0%	-0.267	-1.395
qLogNParEGO	102/150	68.0%	-0.272	-1.530
qLogNEHVI	95/150	63.3%	-0.946	-1.232

Joint improvement makes this difference in budget allocation more concrete. Only 20 of Random’s 150 candidates and 28 of Sobol’s dominated their local default on both objectives, compared with 95–102 for the Bayesian methods. The corresponding shares are 13.3–18.7% and 63.3–68.0%. Figure 6.1 shows that this separation persists at the seed level: Bayesian histories contained 15–21 joint improvements out of thirty, whereas the baselines contained 2–7. A majority of jointly improving candidates can coexist with slightly negative mean throughput differences because the mean also reflects the magnitude of losses among the remaining candidates. The domination share describes frequency, while the mean-quality endpoints retain the magnitudes of both gains and losses.

Sensitivity to practical improvement margins. The strict joint-improvement count includes arbitrarily small differences. Table 6.5 therefore repeats the count with minimum TPS gains of 1%, 2%, and 3%, paired with p99 reductions of 0.5, 1, and 1 ms, respectively. For margins (r, d) , a candidate qualifies when $T_i/\hat{T}_{0,i} - 1 \geq r$ and $L_i - \hat{L}_{0,i} \leq -d$. This post-hoc sensitivity analysis uses the same local controls and all thirty logical slots per seed; it leaves the registered endpoints and tests unchanged.

Table 6.5: Post-hoc joint-improvement sensitivity. Each margin requires both the stated TPS gain and p99 reduction. Strict means $\text{TPS} > 0$ and p99 difference < 0 relative to the local default. Counts retain shared initialization within each method’s logical budget.

Method	Strict	1%, 0.5 ms	2%, 1 ms	3%, 1 ms
Random	20/150 (13.3%)	19/150 (12.7%)	11/150 (7.3%)	9/150 (6.0%)
Sobol	28/150 (18.7%)	24/150 (16.0%)	20/150 (13.3%)	15/150 (10.0%)
qLogNEI	99/150 (66.0%)	93/150 (62.0%)	81/150 (54.0%)	72/150 (48.0%)
qLogNParEGO	102/150 (68.0%)	100/150 (66.7%)	86/150 (57.3%)	79/150 (52.7%)
qLogNEHVI	95/150 (63.3%)	86/150 (57.3%)	78/150 (52.0%)	62/150 (41.3%)

At the 2% TPS and 1 ms p99 margins, the Bayesian methods retain 78–86 qualifying candidates out of 150 (52.0–57.3%), compared with 11 for Random (7.3%) and 20 for Sobol (13.3%). Even the 3% TPS margin retains 62–79 Bayesian candidates against 9–15 for the baselines. The descriptive separation in candidate quality thus persists after small improvements are excluded. These margins express practical effect sizes; they do not estimate the probability that each individual gain exceeds measurement error.

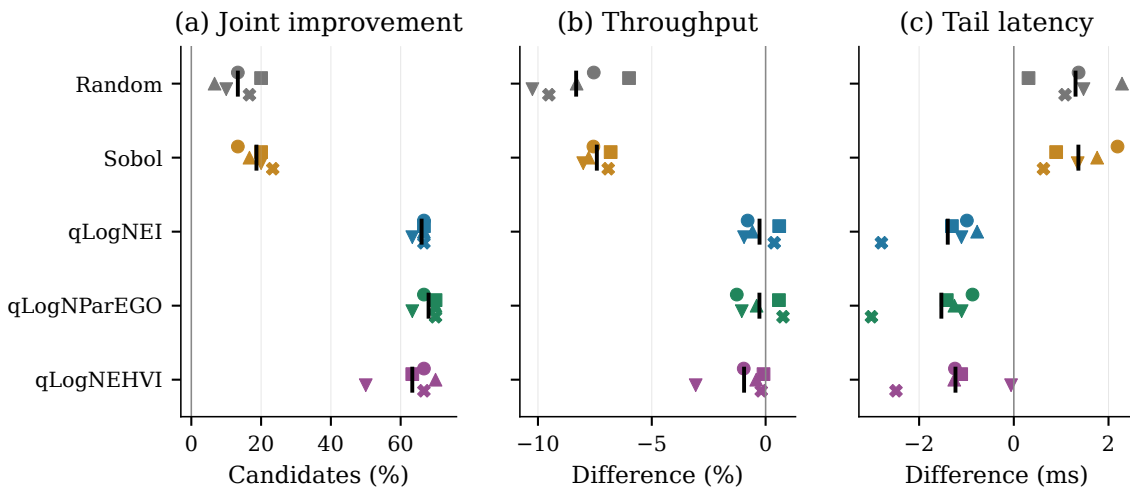


Figure 6.1: Per-seed full-budget candidate quality. Each colored marker represents one seed; black ticks mark five-seed means. Circles, squares, upward triangles, downward triangles, and crosses denote seeds 88408573, 1418705027, 642754166, 1902413987, and 740267717, respectively. Higher values are favorable in (a) and (b), and lower values in (c). Each point summarizes thirty logical candidates.

The common initialization also limits the attribution of these results. Every Bayesian method begins with the same physically measured Sobol points within a seed, and those points contribute to every complete Bayesian budget. The reported shares therefore evaluate the prescribed procedure of initialization followed by acquisition rather than the adaptive phase alone. They cannot be summed across Bayesian methods as counts of unique successful configurations. To distinguish initial coverage from subsequent candidate quality, Table 6.6

decomposes the existing observations into slots 1–12 and 13–30. This is a post-hoc descriptive analysis, not a registered endpoint or a new significance test. It uses the unchanged interpolated local controls, averages each phase within a seed, and then averages equally across the five seeds. A joint improvement requires both higher TPS and lower p99 than the local default. The shared initialization appears once; each adaptive row contains ninety distinct observations, and the baseline rows use the corresponding logical slots.

Table 6.6: Post-hoc candidate-quality decomposition across five seeds. TPS is the mean percentage change from the local default; p99 is the mean difference in milliseconds. Joint improvements are descriptive counts and shares, not independent replications. Slots 13–30 are adaptive only for the Bayesian methods.

Method	Slots	TPS (%)	p99 (ms)	Joint improvements
Shared BO initialization	1–12	−7.153	+0.816	15/60 (25.0%)
Random	1–12	−8.630	+1.581	7/60 (11.7%)
Sobol	1–12	−7.464	+1.305	9/60 (15.0%)
Random	13–30	−8.117	+1.119	13/90 (14.4%)
Sobol	13–30	−7.380	+1.403	19/90 (21.1%)
qLogNEI	13–30	+4.324	−2.869	84/90 (93.3%)
qLogNParEGO	13–30	+4.316	−3.094	87/90 (96.7%)
qLogNEHVI	13–30	+3.193	−2.597	80/90 (88.9%)

The shared initialization contains strong individual observations but has mean control-relative TPS of −7.153%, mean p99 difference of +0.816 ms, and only fifteen joint improvements among sixty observations. In contrast, the adaptive phases average +3.193% to +4.324% TPS and −3.094 to −2.597 ms p99, with 80–87 joint improvements among ninety observations. The corresponding later baseline slots remain below default throughput and above default p99, with only 13–19 joint improvements. The full-budget means combine these phase means with weights 12/30 and 18/30: for qLogNEI, $0.4(-7.153) + 0.6(4.324) = -0.267\%$ after rounding. Near-parity full-budget throughput therefore conceals positive mean throughput in the adaptive portion.

This descriptive decomposition locates the improvement in adaptive slots, conditional on shared initialization and the phases’ different temporal positions. Within the complete prescribed procedure, all Bayesian methods nevertheless have higher mean control-relative throughput and lower mean control-relative p99 than either baseline in every seed. The best-found endpoints provide a complementary assessment of whether this more favorable allocation also expands the frontier.

6.3.2 Best-found Performance and Frontier Development

Table 6.7 summarizes the three raw endpoints at logical slot thirty. Mean best throughput ranges from 3,003.9 TPS for Random to 3,064.2 TPS for qLogNParEGO, with Sobol and the other Bayesian methods between 3,049.6 and 3,059.5 TPS. The Bayesian advantage over Sobol on this endpoint is consequently only 4.1–14.6 TPS, approximately 0.13–0.48% of Sobol’s mean. Mean minimum p99 provides greater separation: the Bayesian values are 25.852–26.138 ms, compared with 26.612 ms for Random and 26.737 ms for Sobol. Even here, the differences are fractions of a millisecond within this controlled workload. Best throughput and minimum p99 may occur at different configurations and must not be interpreted as an attainable joint outcome.

The 0.13–0.48% best-TPS differences are substantially smaller than the 2.93% relative confidence-interval half-width observed in the calibration default block. That half-width de-

scribes uncertainty in the five-observation calibration mean, rather than a transferable per-run noise threshold. Nevertheless, its scale reinforces the weak practical separation of the best-TPS endpoints. The clearer result is the full-budget candidate-quality advantage, including the margin sensitivity in Table 6.5.

Table 6.7: Final raw endpoints as mean $\pm$ sample standard deviation across five seeds. Hypervolume uses the frozen (0, -40) reference and has units TPS $\times$ ms. Throughput and p99 columns report separate best-found extremes.

Method	Hypervolume	Best TPS	Minimum p99 (ms)
Random	40,228 $\pm$ 2,521	3,003.9 $\pm$ 59.6	26.612 $\pm$ 0.611
Sobol	40,426 $\pm$ 847	3,049.6 $\pm$ 30.8	26.737 $\pm$ 0.325
qLogNEI	43,054 $\pm$ 1,183	3,059.5 $\pm$ 34.8	25.926 $\pm$ 0.267
qLogNParEGO	43,343 $\pm$ 659	3,064.2 $\pm$ 28.5	25.852 $\pm$ 0.163
qLogNEHVI	42,323 $\pm$ 1,733	3,053.7 $\pm$ 37.3	26.138 $\pm$ 0.437

Figure 6.2 locates these endpoints within the search budget. The three Bayesian curves coincide through slot twelve because their initialization observations are shared. At that point, their mean hypervolume is 40,983.5; the eighteen adaptive slots increase it to 43,053.6 for qLogNEI, 43,343.4 for qLogNParEGO, and 42,322.7 for qLogNEHVI. Over the same adaptive portion, mean best throughput rises from 3,038.2 TPS by only 15.4–26.0 TPS, while mean minimum p99 falls from 26.514 ms to 25.852–26.138 ms. The curves thus show continued frontier improvement after initialization, with limited additional movement in maximum throughput. They are indexed by logical opportunities, not elapsed time, and do not establish which method would achieve its result first under an equal wall-clock budget.

Mean final hypervolume is 40,228.2 for Random and 40,426.2 for Sobol, compared with 42,322.7–43,343.4 for the Bayesian methods. qLogNParEGO has the largest mean, exceeding Random by 3,115.2 and Sobol by 2,917.1 in the frozen units. This aggregate ordering is not stable enough to identify a universally preferred acquisition. For example, qLogNEHVI has the highest hypervolume in seed 1418705027, while qLogNEI leads in seed 1902413987 and qLogNParEGO leads in seed 740267717. A post-hoc leave-one-seed-out check also reverses parts of the ordering: omitting seed 642754166 makes qLogNEI rather than qLogNParEGO best on mean minimum p99, while omitting seed 1902413987 puts qLogNEHVI ahead of qLogNEI on mean hypervolume, although qLogNParEGO retains the hypervolume lead in every omission. The throughput-only acquisition also improves the two-objective frontier despite using only throughput in its acquisition objective. Under this workload, useful throughput regions can contain favorable latency outcomes, but the experiment does not establish that a throughput-only objective preserves the same trade-offs in other conditions.

The pooled view in Figure 6.3 provides the corresponding configuration-level geometry. Eight of the 630 unique physical candidates are nondominated: three qLogNEI observations, two qLogNParEGO observations, two qLogNEHVI observations, and one shared initialization observation. Their extremes run from 3,036.2 TPS at 25.557 ms to 3,113.7 TPS at 26.242 ms. The latter point belongs to shared initialization, so the largest raw throughput in the entire cohort cannot be credited to a particular adaptive acquisition. Pooling exposes the observed trade-off but combines measurements from different temporal conditions and seeds. This frontier is therefore descriptive; the paired seed-level endpoints remain the basis for method comparison.

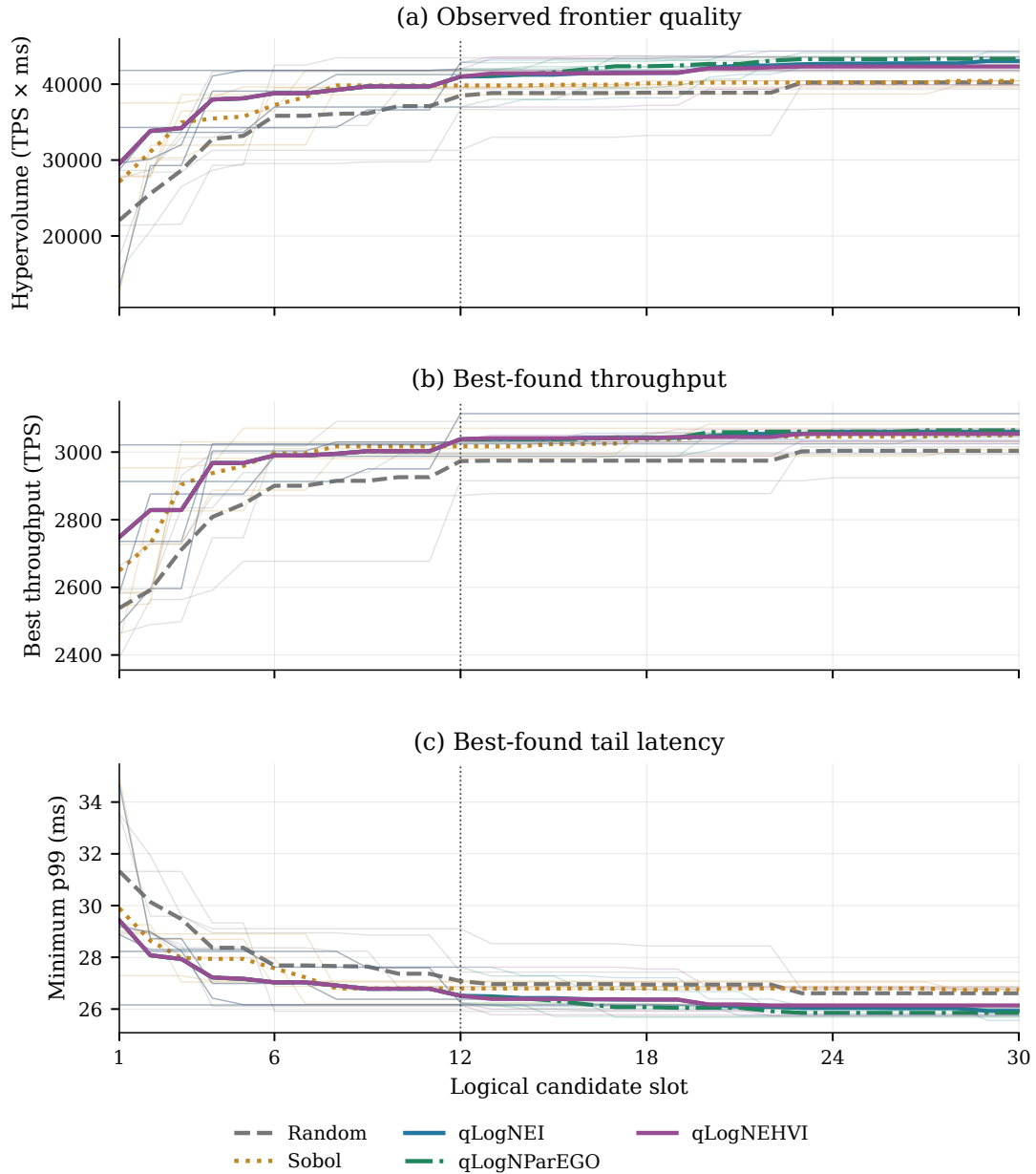


Figure 6.2: Search trajectories for all five methods. Thin lines show individual seeds and bold lines their arithmetic mean at each slot ($n = 5$). The vertical line marks the end of the twelve shared Bayesian initialization slots. Hypervolume and best throughput increase with improvement; minimum p99 decreases. The individual-seed lines are descriptive trajectories, not confidence bounds.

6.3.3 Post-hoc Surrogate Diagnostic

To examine which input directions the specified surrogate treats as more variable, Table 6.8 summarizes fresh fits to each Bayesian method’s final thirty-observation history. The diagnostic uses the saved normalized vectors, raw objectives, and model specification in Table 4.2, with a fixed fitting seed of 20260922. It comprises fifteen TPS models and ten negative-p99 models across five seeds. These are post-hoc terminal fits, distinct from the online fits that used twelve through twenty-nine observations to select candidates.

The smallest median length scale belongs to `max_wal_size` for both TPS (0.25) and negative p99 (0.20), consistent with its strong screening score. The TPS median for `shared_buffers` is 1.35, compared with 2.41–4.10 for the remaining inputs; for latency,

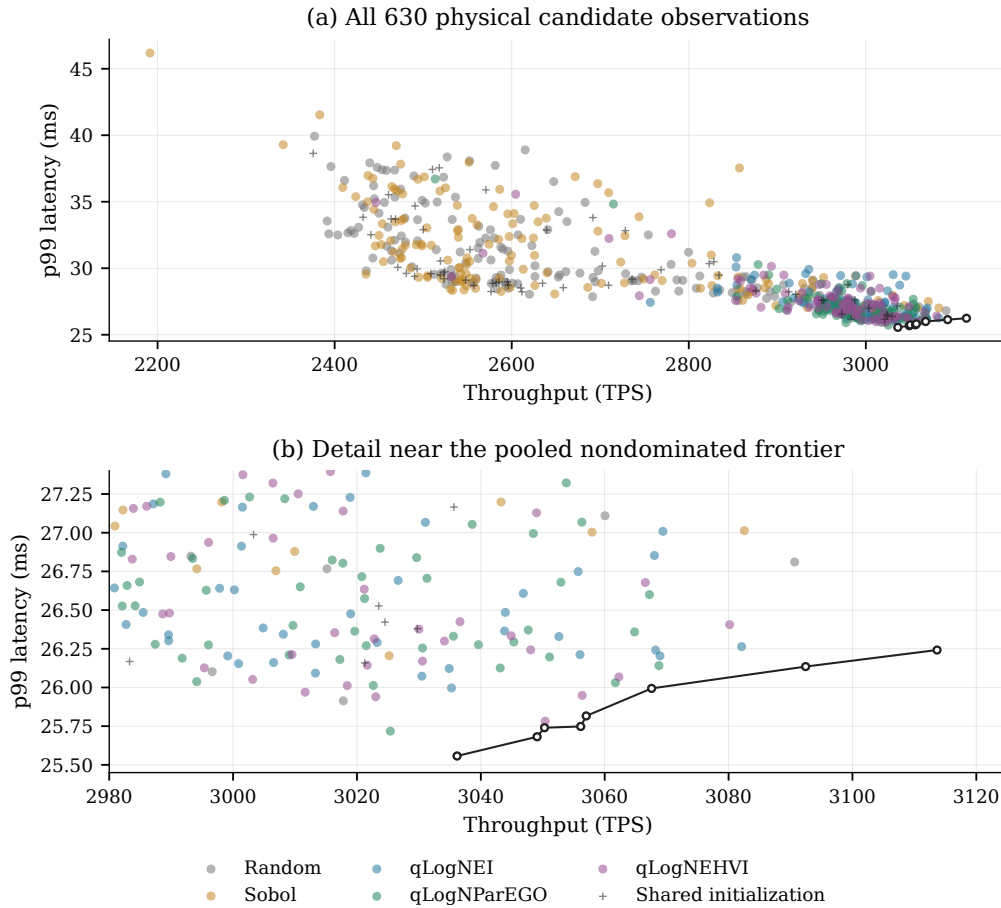


Figure 6.3: Raw physical candidate outcomes across five seeds. Panel (a) includes all 630 candidates; panel (b) enlarges the frontier region. Shared initialization is drawn once, and the black line joins the eight pooled nondominated observations. Higher throughput and lower p99 are favorable. Controls are excluded from this candidate frontier; pooled points do not supply independent method replications.

Table 6.8: ARD length scales in normalized input units: median [minimum, maximum] across fifteen TPS and ten negative-p99 terminal models. Smaller values indicate more rapid fitted variation along the corresponding coordinate.

Parameter	TPS length scale	Negative-p99 length scale
shared_buffers	1.35 [0.68, 3.04]	0.88 [0.34, 1.57]
effective_cache_size	4.01 [0.79, 6.83]	2.68 [0.26, 5.04]
work_mem	2.94 [0.63, 5.70]	0.94 [0.24, 5.19]
checkpoint_timeout	3.40 [0.53, 6.90]	2.07 [1.10, 3.71]
checkpoint_completion_target	2.41 [0.25, 6.70]	1.19 [0.32, 1.68]
max_wal_size	0.25 [0.15, 0.51]	0.20 [0.11, 1.45]
random_page_cost	4.10 [0.83, 5.99]	1.85 [0.32, 3.97]
max_parallel_workers_per_gather	2.47 [0.29, 6.10]	0.87 [0.19, 4.20]

several coordinates have similar medians near one. The ranges show substantial variation across the small adaptive histories. ARD therefore supplies a model-conditional indicator of sensitivity within the sampled domain, not a causal importance ranking or an out-of-sample accuracy assessment. Predictive validation on independent configurations remains a separate evaluation requirement.

6.3.4 Temporal Drift and Per-seed Variability

The raw endpoints must be interpreted alongside the changing default measurements in Figure 6.4. Two seeds exceeded registered transparency thresholds. Seed 88408573 had a fitted throughput change of -165.056 TPS, equivalent to 5.75% of its control mean and above the 5% limit; its fitted p99 increase of 3.429 ms remained below the latency limit. Seed 740267717 had a fitted p99 increase of 9.002 ms, exceeding the 5 ms limit, while its throughput change remained below 5%. The other three seeds were unflagged. These outcomes classify the completed experiment as complete with drift flags, retaining all observations under the frozen analysis policy.

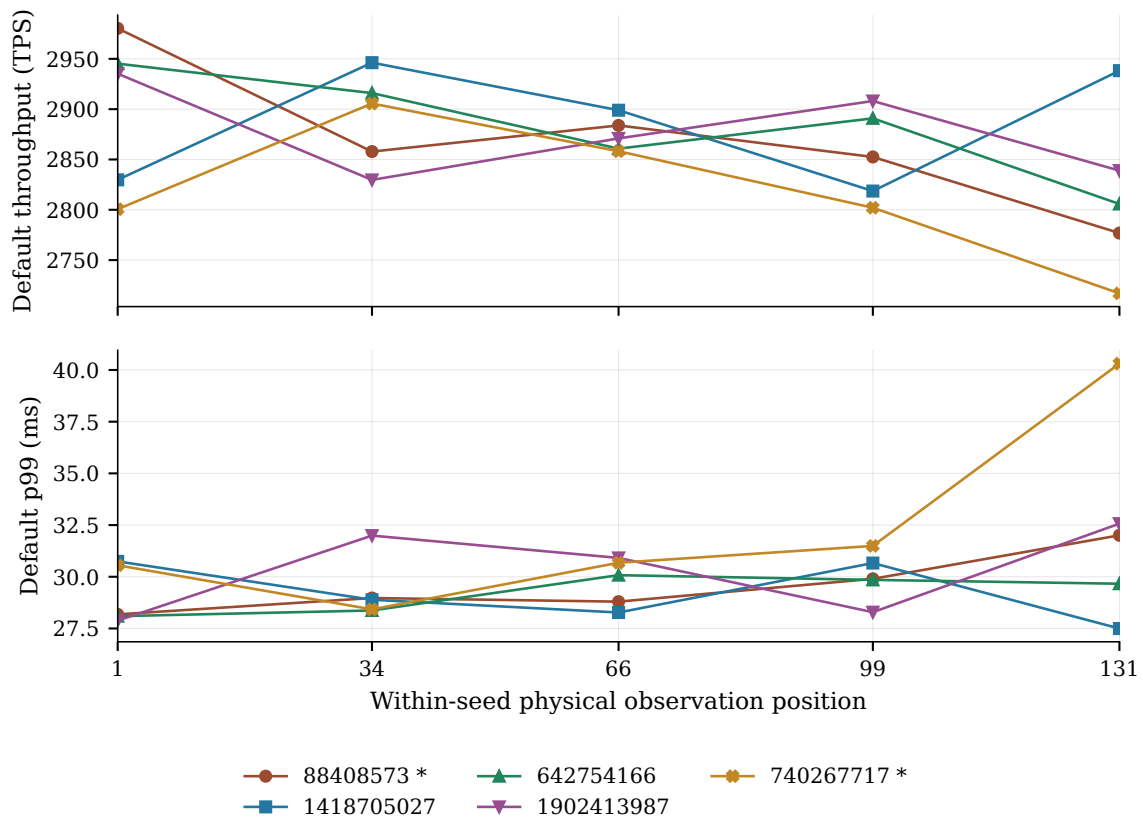


Figure 6.4: All five default controls in each primary seed, shown at common within-seed physical positions. Lines join the measured controls rather than fitted regressions. Asterisks identify seeds exceeding either registered drift threshold; no seed is excluded. These controls support the local interpolation used by the full-budget relative endpoints.

The controls clarify why raw and control-relative results serve different purposes. In seed 740267717, the final default p99 reached 40.311 ms, making the interpolated reference sensitive to a pronounced late increase. The Bayesian mean p99 differences in this seed range from -3.007 to -2.488 ms, larger reductions than in the other seeds. Local adjustment therefore exposes performance relative to the measured default while also inheriting uncertainty about

what occurred between its five observations. Raw outcomes and local comparisons are therefore retained together; Section 7.5 discusses the residual temporal uncertainty.

Variability also occurs within the Bayesian family. In seed 1902413987, qLogNEHVI’s mean p99 difference is only -0.058 ms and fifteen candidates jointly improve on the local default, compared with nineteen for each of the other Bayesian methods. Its final hypervolume is 39,914.1, close to 39,899.6 for Random, although its full-budget candidate quality remains better than both baselines. Such cases show why a mean frontier ranking does not fully describe the reliability of search. The more consistent finding is the separation of Bayesian and baseline full-budget quality, with acquisition-level differences depending more strongly on the seed.

6.3.5 Endpoint Uncertainty and Ranking Boundaries

Table 6.9 retains the registered whole-seed bootstrap intervals for the two full-budget endpoints. The intervals place Random and Sobol on the throughput-loss side of the default and the Bayesian methods near parity, while their p99 intervals lie on opposite sides of zero. These descriptive intervals preserve the five paired seed-level units and summarize effect scale alongside the registered paired comparison.

Table 6.9: Registered 95% whole-seed bootstrap intervals for mean full-budget endpoints ($n = 5$). Intervals summarize limited seed-level evidence and do not imply an independent-candidate sample size.

Method	R^T interval (%)	D^L interval (ms)
Random	$[-9.565, -7.013]$	$[+0.734, +1.861]$
Sobol	$[-7.826, -7.001]$	$[+0.878, +1.850]$
qLogNEI	$[-0.809, +0.276]$	$[-2.120, -0.925]$
qLogNParEGO	$[-1.004, +0.460]$	$[-2.275, -1.027]$
qLogNEHVI	$[-2.053, -0.196]$	$[-1.967, -0.535]$

All six Bayesian-versus-baseline contrasts in each control-relative endpoint family attain an unadjusted exact paired permutation p-value of 0.0625 and a Holm-adjusted value of 0.625. The corresponding Cliff’s delta is $+1$ for throughput and -1 for p99, indicating complete separation of the observed seed-level values in their favorable directions. As established in Equation 5.4, 0.0625 is the smallest possible two-sided p-value with five paired seeds; even complete directional agreement cannot meet a 0.05 significance threshold.¹ No contrast establishes conventional statistical superiority under this procedure, and the correlated endpoints are not independent confirmations. The results support a bounded descriptive conclusion about more favorable full-budget allocation under the evaluated conditions, with no unique acquisition winner established.

6.4 Multi-fidelity and Confirmatory Results

6.4.1 Retrospective Prefix Evaluation

The retrospective analysis uses the fixed three-seed population specified in Section 5.6: 378 unique physical candidates and fifteen controls. Table 6.10 summarizes the two prefix lengths. All per-seed throughput-rank correlations exceeded 0.80, with stronger agreement for 120 s than

¹Had a favorable direction been registered in advance, the corresponding one-sided exact p-value for these contrasts would be $1/2^5 = 0.03125$ (approximately 0.031). Holm correction over all ten method pairs within each endpoint family gives 0.3125 for these contrasts, still above 0.05. The registered two-sided analysis is retained; this hypothetical alternative changes none of the inferential claims.

for 60 s. Neither policy rejected any of the 96 candidates belonging to the per-seed full-fidelity top quartiles. Both policies passed the per-seed ranking and false-rejection gates and reduced pooled modeled time under the implemented analyzer. The 120-second policy saved no time in seeds 88408573 and 642754166, so it did not satisfy the manifest’s stronger per-seed cost wording. Both policies promoted almost every candidate. The resulting selectivity, rather than rank agreement alone, determined their small time savings.

Table 6.10: Retrospective prefix evaluation on 378 unique candidates from three seeds. Correlation ranges summarize per-seed Spearman correlations; rejection counts and modeled savings are pooled descriptive totals, with controls and lifecycle overhead included in the cost denominator.

Quantity	60 s prefix	120 s prefix
Per-seed rank correlation	0.815–0.883	0.940–0.964
Promoted candidates	370/378	377/378
Rejected candidates	8	1
Top-quartile false rejections	0/96	0/96
Modeled time saved	1.200 h	0.133 h
Modeled lifecycle saving	0.660%	0.073%

At 60 s, eight rejections saved 540 measurement seconds each, giving 4,320 s in total. At 120 s, the single rejection saved 480 s. The longer prefix improved ranking agreement but reduced both the number of rejected candidates and the saving available from each rejection. Because restoration, restart, and warm-up had already been paid, these measurement reductions translated into only 0.660% and 0.073% of modeled lifecycle cost. The zero false-rejection count concerns the registered throughput top quartile and uses retrospective complete-run validity and bracketing controls.

6.4.2 Live Promotion and Runtime Savings

The live demonstration completed all thirty fixed Sobol candidates and five causal default controls with no benchmark failures. Twenty-nine candidates passed the 60-second promotion gate and continued for another 540 s. The remaining candidate had a throughput-to-control ratio of 0.780488, below the frozen 0.80 threshold, and stopped after its prefix. This execution demonstrated that the decision and continuation mechanism operated with the preceding control available at the decision point. Its thirty candidates remain a separate demonstration population, not further Sobol observations in the primary experiment.

Successful lifecycle time was 15.655 h, increasing to 15.707 h when one retained pre-measurement infrastructure failure is included. Relative to the matched all-F3 counterfactual of 57,084.821 s, the total including the failed attempt was 56,544.821 s, saving 540 s, or 0.150 h and 0.946%. The same failed-attempt cost is retained on both sides of this comparison. Separately, the runtime model’s absolute relative error for successful lifecycle time was 2.303%, within the registered 10% limit. The observed lifecycle supports the accounting model and demonstrates a 0.946% saving against its matched counterfactual. The rejected candidate’s full-F3 outcome remains unobserved.

6.4.3 Finalist Confirmation and Configuration Selection

Confirmation repeated the four fixed finalists from the first three primary seeds under the four common-seed blocks defined in Section 5.7. All twenty observations, including the four central default controls, completed validly on their first attempt. There were no candidate or infrastructure failures, and successful lifecycle time totaled 9.176 h. Table 6.11 reports the configuration

means and same-block default differences. Every finalist satisfied the practical eligibility gates, so the frozen latency-first rule selected configuration E, whose mean p99 of 25.952 ms was the lowest among the four. Table A.1 gives its complete requested settings in native units.

Table 6.11: Finalist confirmation across four common-seed blocks. Relative endpoints compare each finalist with its same-block default before averaging. All finalists passed the practical gates; E was selected by the frozen latency-first rule.

Finalist	Mean TPS	Mean p99 (ms)	R^T (%)	D^L (ms)
T	3042.5	26.255	+5.470	-3.086
L	3017.2	26.782	+4.593	-2.559
H	3036.3	26.793	+5.256	-2.548
E	3032.3	25.952	+5.118	-3.389

E maintained mean throughput of 3,032.3 TPS, approximately 99.7% of T’s highest finalist mean of 3,042.5 TPS. Relative to the same-block default, E improved throughput by 5.118% and reduced p99 by 3.389 ms on average. Its p99 was the lowest in three of the four blocks; T had lower p99 in the remaining block. The mean difference between E and T was only 0.303 ms, so the selection represents the defined practical rule rather than an invariant ordering of individual measurements. The positive throughput contrasts here describe repeated selected configurations and therefore do not conflict with the near-parity averages over complete Bayesian search budgets. Selection and full-budget quality address different populations and different uses of the observations.

E sets `shared_buffers` to 182,848 blocks, approximately 1.395 GiB or 34.88% of the 4 GiB limit. Its position within the permitted interval is $(182848 - 16384) / (183500 - 16384) = 0.9961$, leaving only 652 blocks (5.09 MiB) below the cap. This proximity makes a boundary-limited optimum plausible, but the observations provide no response measurements beyond the cap or controlled one-parameter comparison establishing a continuing gain. The selected result is consequently conditional on the operational domain justified in Section 5.4. A targeted boundary study could compare the current value, the cap, and prospectively approved larger allocations while holding E’s other settings fixed and monitoring memory headroom under the same restored blocking procedure.

The four-block recommendation concerns the selected configuration E. It was subsequently applied to the research target under separate operator authorization. The recorded post-application verification found the target healthy, the active settings matching the selected configuration, and no pending restarts or active campaigns. This completes the operational configuration-application record.

Chapter 7

Discussion

Key findings. The framework completed 655 valid physical observations in 303.795 h while preserving candidate identity through two infrastructure retries. Bayesian search produced better full-budget control-relative TPS and p99 than Random and Sobol in every seed, retaining 52.0–57.3% jointly improving candidates at margins of 2% TPS and 1 ms p99, against 7.3–13.3% for the baselines. Best-throughput differences between Bayesian search and Sobol were only 0.13–0.48%, leaving candidate allocation as the clearer distinction. Prefix promotion preserved retrospective throughput rankings but saved less than 1% of total lifecycle time. Finally, four-block confirmation selected E with mean gains of 5.118% TPS and 3.389 ms p99 relative to the reset default.

7.1 Reliability of Measurement and Budget Accounting

Execution completeness, auditability, and offline report reproduction provide complementary evidence of measurement reliability. All 655 physical observations completed validly and passed both restoration fingerprint tiers. Recovery preserved the assigned candidate and workload seed through two infrastructure failures, including one interruption requiring supervised reconciliation. The retained failures and drift flags make the record useful for assessing both execution and measurement conditions. Together, these results demonstrate a recoverable research workflow with explicit accounting for the conditions under which its observations were produced.

That accounting is essential because equal logical budgets have a different meaning from physical execution cost. The shared initialization supplies equal starting information to the Bayesian methods while reducing the physical work needed to construct their thirty-candidate histories. Counting each shared retry once per method would overstate infrastructure failures, as Table 6.2 illustrates. Keeping logical attribution, physical observations, and attempts separate therefore protects the interpretation of both search effectiveness and reliability. The corrected reporting record also shows why byte-identical reproduction should be accompanied by checks of what each reported count represents.

Restoration controls the dataset on which this comparison depends, while interleaved defaults expose residual performance variation. The successful fingerprints and retained drift flags address these complementary properties: verified state and observed temporal stability. Their combined cost is substantial, with restoration consuming 26.7% of successful primary attempt time. This makes lifecycle accounting part of the practical assessment of tuning, especially when a proposed shortcut acts only after preparation. The limits of temporal adjustment and independent replication are collected in Section 7.5.

7.2 Full-budget Effectiveness and Objective Trade-offs

Full-budget candidate quality provides the strongest evidence for effective budget allocation. Each Bayesian method has better mean control-relative TPS and p99 than both baselines in every seed. This matters because a search budget purchases a sequence of evaluations, including configurations that will never be selected for continued use. The margin analysis in Table 6.5 shows that the separation survives the exclusion of small improvements. The adaptive portion further averages 3.193–4.324% above local-default TPS, while initialization losses bring the full-budget means back toward parity. These observations give descriptive support to RQ1’s proposition that model-based selection uses the available evaluation budget more effectively under the measured conditions.

The corresponding registered tests remain descriptive in this design: their minimum two-sided p-value is 0.0625, and the relevant Holm-adjusted values are 0.625. Section 5.5 relates this resolution to the approximately 300 h execution budget and specifies replication requirements for a future confirmatory comparison. Effect magnitude and agreement across seeds therefore carry the interpretation here. The candidate-level sensitivity counts strengthen the practical description without adding independent replications to the five-seed experiment.

Best-found outcomes describe a different use of that budget. Bayesian mean best throughput exceeds Sobol by only 0.13–0.48%, well below the 2.93% calibration interval half-width, whereas their mean final hypervolume and minimum p99 show clearer separation. A maximum retains only the most favorable observation; a full-budget mean also includes costly poor choices. Sobol can therefore locate a strong final configuration while sampling many degrading ones. The common Bayesian initialization already includes the cohort’s highest-TPS observation, explaining why subsequent adaptation is more visible in candidate quality than in the throughput maximum.

Objective geometry also helps explain why throughput-only qLogNEI achieves favorable latency and hypervolume. The sampled high-throughput region overlaps with low-p99 configurations, allowing a throughput-directed search to improve both outcomes in this closed-loop workload. Conversely, qLogNEHVI scores candidates by predicted hypervolume improvement, whereas the final ranking uses their realized noisy outcomes. Eighteen adaptive decisions and finite-pool acquisition maximization leave considerable scope for seed-dependent ordering. The terminal GP diagnostic identifies `max_wal_size` as the direction with the shortest median fitted length scale, but its variable estimates support retaining a model-conditional interpretation of parameter sensitivity.

7.3 Multi-fidelity and Operational Implications

Accurate prefix rankings produced little end-to-end acceleration because almost every candidate was promoted. The retrospective 60-second policy rejected eight of 378 candidates, while the live policy rejected one of thirty. Restoration, activation, and warm-up were already paid before either decision, and promoted candidates still received 600 measurement seconds. The longer 120-second prefix improved ranking agreement but rejected only one candidate and saved fewer seconds on that rejection. Its modeled lifecycle saving was therefore smaller. Useful early stopping requires both adequate discrimination and a substantial avoidable portion of lifecycle cost.

The duration pilot and prefix analysis assess different properties of a short measurement. The pilot checks agreement in objective values, including p99, while the promotion policy uses throughput to decide whether to continue. A prefix can preserve a useful ranking while es-

timating tail latency too imprecisely to replace the full measurement. Retrospective ranking evidence and the live operational demonstration consequently complement each other. The cost-gate discrepancy is recorded with the other protocol exceptions in Table 6.1; the remaining secondary-population limits are consolidated in Section 7.5.

Four-block confirmation turns an exploratory candidate into a repeated configuration choice. E has the lowest mean p99 while retaining approximately 99.7% of T’s mean throughput, satisfying the latency-first eligibility rule. Its same-block differences of +5.118% TPS and -3.389 ms p99 give the practical configuration result, although the mean E–T latency separation is only 0.303 ms. E’s buffer allocation is also close to the search-space ceiling, making the bounded-domain interpretation and proposed boundary check in Section 6.4.3 relevant to reuse. Candidate generation, repeated evaluation, and verified activation thus answer successive operational questions, with confirmation supplying the performance evidence for selection.

7.4 Relationship to Prior Work and Practical Implications

The contrast between intermediate candidate quality and best-found outcomes connects to CG-PTuner’s distinction between cumulative reward during online tuning and iterative best performance during offline tuning [6]. Here, restored observations provide a corresponding comparison of full-budget quality and final search outcomes. Their practical lesson is to assess the quality of the configurations encountered along the way, while recognizing that this restored benchmark measures candidate quality rather than accumulated service impact during online adaptation.

The common search space and isolated histories also follow the modular perspective of Zhang et al., who distinguish knob selection, optimization, and knowledge transfer [4]. Fixing the first and third components makes the present acquisition comparison interpretable. Otter-Tune instead uses historical observations and workload mapping to accelerate tuning [1], so its information budget differs from the shared initialization and isolated adaptive histories here. Kanellis et al.’s findings on influential-knob selection further motivate a tractable configuration space [18]. The exploratory screening amendment limits the validation of this particular subset, while leaving a common domain for the five methods.

The observed drift and small best-found differences make the operational concerns identified by Van Aken et al. relevant to interpretation [25]. Reporting controls, raw outcomes, failures, and restoration costs together gives the evidence needed to evaluate a small gain before adopting a configuration. Similarly, multi-fidelity optimization relies on approximations that are both informative and cheaper [21]. In this lifecycle, preparation precedes the fixed promotion rule and dominates much of its avoidable cost. Improvements to restoration or warm-up therefore merit evaluation alongside more selective prefix policies, under the same state-verification requirements.

7.5 Threats to Validity and Limitations

Internal validity is chiefly limited by temporal variation between the five controls in each seed. Interpolation follows physical position and can miss abrupt or elapsed-time-dependent changes; the late p99 increase in seed 740267717 particularly affects its relative latency gains. The optimizers train on raw histories, so analysis-time control adjustment leaves uncertainty about drift’s influence on adaptive choices. Restoration verifies dataset state while host-cache effects and background interference remain possible. The protocol exceptions in Table 6.1, especially the screening amendment, further delimit the evidential basis of the final design.

Construct validity is bounded by the closed-loop workload and its successful transactions. These TPS and p99 outcomes are distinct from application response times under an independently imposed arrival rate or a latency service requirement. Full-budget means, joint-improvement shares, extremes, and hypervolume summarize different properties of the same observations. Measurement noise can favor selected extremes, and the pooled frontier combines different temporal conditions. The practical margins expose sensitivity to effect size; repeated F4 measurements address selection variability for the four finalists.

Statistical conclusions rest on five seed-level units. Adaptive dependence and shared initialization preclude treating candidate rows as independent replications, while correlated endpoints provide related evidence. Test resolution and multiplicity prevent a conventional superiority conclusion; small-sample bootstrap intervals neither repair that limitation nor establish equivalence. Continuation was informed by the first execution block, with the pooling rules fixed before the second. The GP length-scale diagnostic is a post-hoc fit to selected histories and provides no independent validation of predictive accuracy or causal parameter importance.

External validity is restricted to PostgreSQL 18.1, one host and 4 GiB target, one synthetic operating point, and the frozen eight-knob domain. The finite proposal pool, twelve-point initialization, and eighteen adaptive slots define the implemented search comparison; their separate effects were not isolated. E's proximity to the buffer ceiling leaves performance beyond that boundary unmeasured. Comparisons with expert tuning, transfer-based systems, reinforcement learning, index selection, and schema optimization are also outside the measured evidence. Independent clean-machine replication remains outstanding.

The reset default allocates only 128 MiB to `shared_buffers`, or 3.125% of the target memory. Consequently, E's 5.118% TPS advantage measures improvement over that reproducible reference, while the gain over a practical heuristic configuration remains unknown. A 25% allocation would be 1 GiB, consistent with PostgreSQL's starting guidance [7]. Neither that configuration nor a PGTune recommendation was evaluated in the F4 blocks. Such a baseline is needed to distinguish the benefit of basic resource sizing from the incremental value of learned tuning.

Secondary evidence has narrower populations: retrospective prefixes cover three primary seeds and define promise by throughput, while a live rejected candidate lacks the full-F3 outcome needed to assess false rejection. F4 repeats four selected configurations in four blocks and supplies configuration-level evidence. Operational activation was verified through settings and health checks, without a subsequent performance comparison. Long-term production benefit and autonomous operational safety therefore remain open evaluation questions.

Chapter 8

Conclusion and Future Work

8.1 Conclusions

This thesis developed and evaluated a framework for PostgreSQL configuration tuning in which candidate selection is supported by a controlled, recoverable, and auditable measurement process. Its central contribution is the integration of that process with a fixed-budget comparison of established Bayesian, random, and Sobol search methods. Verified configuration activation, database restoration, retained failure records, and explicit accounting of shared observations make the experimental budget and its outcomes interpretable. This foundation supports an assessment of search quality across all evaluated candidates, alongside the best configurations and throughput–latency trade-offs found by each method.

The five-seed comparison provides descriptive support for the proposition that model-based search uses a limited candidate budget more effectively than undirected search under the evaluated conditions. The Bayesian methods’ more favorable full-budget quality coexisted with closely matched best-found throughput, showing why the strongest observed configuration alone gives an incomplete account of tuning effectiveness. However, five paired seeds do not permit conventional statistical confirmation under the registered exact tests, and the observed acquisition rankings vary across seeds. The contribution is therefore a demonstrated difference in how the tested methods allocated their budgets, with explicit evidential limits, rather than a general ranking of Bayesian acquisition functions.

The secondary evaluations extend this conclusion from search to measurement cost and configuration choice. Prefix-based promotion operated successfully, but its lifecycle savings remained below 1% because few candidates were rejected and preparation costs had already been incurred. Separate finalist confirmation (F4) selected configuration E, which averaged 5.1% higher throughput and 3.4 ms lower p99 than the same-block default across four failure-free blocks. These gains concern repeated measurements of a selected configuration; they are compatible with near-parity throughput averaged over complete Bayesian search budgets, which include exploratory candidates. Confirmation supports the practical latency-first choice while preserving uncertainty about small differences between finalists. Together, these results establish a bounded tuning procedure that connects candidate generation, trustworthy measurement, and repeated configuration assessment. They also show that useful search behavior, reduced experimental cost, and repeatable final performance require distinct evidence.

8.2 Answers to the Research Questions

For **RQ1**, Bayesian search allocated its budget more consistently to favorable candidates than Random and Sobol in all five observed seeds. The full-budget control-relative throughput and p99 comparisons agreed in direction across every Bayesian-versus-baseline contrast, with Bayesian mean throughput near default parity and lower mean p99. This supports the value of adaptation beyond a single best-found result, but it does not establish a general throughput gain over the default. As developed in Section 7.2, the evidence supports the research proposition descriptively without establishing statistical superiority or equivalence between methods.

For **RQ2**, the framework preserved complete and auditable experimental execution within the research environment. All 655 scheduled physical observations completed with valid measurements across 657 attempts, while the two infrastructure failures and two seed-level drift flags remained in the record. The interrupted measurement required operator-assisted reconciliation before retry. Separate logical and physical accounting preserved thirty valid candidate observations per method and seed without counting shared Bayesian initialization as independent measurements. The resulting evidence supports recovery, traceability, and offline report reproduction; independent clean-machine replication remains unestablished (Section 7.1).

For **RQ3**, the Bayesian methods achieved higher mean final hypervolume and lower mean minimum p99 than the baselines, while their mean best-throughput advantage over Sobol was only 0.13–0.48%. qLogNParEGO had the largest mean hypervolume, but no acquisition led consistently across seeds and objectives. The favorable frontier outcomes of throughput-oriented qLogNEI also show that explicit multi-objective acquisition was not necessary to find useful latency trade-offs in this workload. These findings characterize the tested search space and budget; they do not establish that throughput optimization generally captures tail-latency preferences or that one acquisition is universally preferable (Section 7.2).

For **RQ4**, short prefixes preserved throughput ranking sufficiently to pass the retrospective per-seed ranking and false-rejection gates, but did not materially accelerate the evaluated lifecycle. Modeled savings were 0.660% for the 60-second policy and 0.073% for the 120-second policy, while the live 60-second demonstration saved 0.946%. The retrospective cost gate used pooled savings; at 120 s, two seeds saved no time and therefore did not meet the manifest’s stronger per-seed cost condition. No retrospective top-quartile throughput candidate was rejected, but the live rejected candidate lacks the full-fidelity outcome needed to assess that decision. The answer is consequently positive for the tested retrospective ranking criterion and operational promotion mechanism, but negative for substantial cost reduction, with general rejection safety unresolved (Section 7.3).

For **RQ5**, confirmation provided sufficient repeatability for the predefined practical selection, although the finalist order was not invariant. All four finalists passed the eligibility gates, and configuration E achieved the lowest mean p99 while retaining approximately 99.7% of the highest finalist mean throughput. E had the lowest p99 in three of four blocks; its subsequent application to the research target verified settings and operational health without adding a performance comparison. Confirmation therefore supports the selected configuration within its evaluation conditions, without establishing superiority of its originating acquisition method or sustained post-application benefit (Section 6.4.3).

8.3 Limitations

The main empirical limitation is the combination of a narrow operating environment and few independent replications. The comparison covers one PostgreSQL version, one host and re-

source envelope, one synthetic workload point, and an eight-knob space selected through an exploratory screen affected by drift. Five paired seeds constrain inference even though the experiment contains hundreds of physical observations; adaptive candidates and shared initialization do not supply additional independent replications. Temporal interleaving and default controls expose variation but cannot eliminate unobserved changes between controls or their influence on adaptive histories. Accordingly, the observed effectiveness cannot be assumed to transfer to other workloads, hardware, bounds, or tuning budgets.

The secondary evidence is narrower still. Retrospective promotion evaluates a throughput-based criterion on three seeds, and the live demonstration does not establish the full-fidelity quality of its rejected candidate. Confirmation compares four selected configurations in four blocks rather than independently replicated search methods. Neither the primary comparison nor confirmation establishes application-level performance under production arrivals, autonomous operational safety, or long-term benefit after activation. These boundaries, detailed in Section 7.5, limit the claims to an auditable research workflow and the measured outcomes of its defined experimental populations.

8.4 Future Work

The first priority is to test whether the full-budget advantage persists with more independent seeds and across additional operating conditions. A larger paired design should determine its replication count in advance from the desired effect resolution and multiplicity procedure, while retaining shared initialization and physical-cost accounting explicitly. Extending the evaluation to different workload mixes, concurrency levels, hardware, and resource allocations would distinguish repeatable search behavior from advantages specific to the current configuration space. Repeating knob screening under stronger drift control would also permit a better-supported assessment of the selected bounds. Comparisons of initialization sizes, proposal pools, and adaptive budgets could then isolate which implementation choices influence candidate quality and acquisition ordering.

Improving measurement efficiency requires addressing the complete lifecycle. Alternative restoration mechanisms and warm-up procedures should be evaluated against the same state-verification and objective-agreement requirements before their lower cost is treated as an improvement. More selective promotion policies could be assessed using causal controls and full-fidelity audit measurements for a sample of rejected candidates, making false-rejection risk observable. Such evaluation should consider latency extremes and Pareto-front preservation as well as throughput ranking. Cost-aware fidelity selection would then have an empirical basis that includes preparation overhead, decision quality, and the actual opportunities for avoiding work.

A further extension is to investigate temporal variation within the tuning decision itself. More frequent or adaptively scheduled controls, combined with time-aware surrogate models, could be compared with the current approach to determine whether modeling drift improves candidate selection enough to justify its additional measurement cost. Repeated finalist evaluation across longer periods and changing workloads would separately test whether practical configuration choices remain useful after their initial confirmation. Application-level experiments would be needed before extending those findings to a continuously operating service. Independent reproduction from an archived software and evidence release would complete the corresponding assessment of portability, testing whether the framework’s measurement and accounting contract can be maintained beyond the original research environment.

Appendices

Appendix A

Selected PostgreSQL Configuration and Encoding

A.1 Confirmed Configuration and Native Units

The bounds, parameter domains, and default control values are specified in Table 5.3. Within that space, the confirmatory procedure selected configuration E under the latency-first rule described in Section 5.7. Table A.1 records its complete requested settings in codec order, connecting the configuration-level results to the native values used for activation. The selection applies to the evaluated workload and resource envelope; it does not establish a general recommendation for other installations.

Table A.1: Requested settings of confirmed configuration E in codec order. Units follow the native configuration representation.

Parameter	Requested value	Unit
shared_buffers	182,848	8 kB blocks
effective_cache_size	465,506	8 kB blocks
work_mem	18,022	kB
checkpoint_timeout	1,430	s
checkpoint_completion_target	0.936696	–
max_wal_size	633	MB
random_page_cost	3.655316	–
max_parallel_workers_per_gather	3	workers

The finalist manifest, `v2/config/f4-confirmation.json`, binds E to source primary run

`e6d70fab-0c90-5795-96a1-5021bff1194c`.

The authenticated F4 analysis records the selected vector, and the application record links it to the research target. Requested real-valued settings retain the codec’s six-decimal representation; PostgreSQL may display them with lower precision, so activation checks use the representation tolerance described in Chapter 4. The search-space specification remains in `v2/config/primary-comparison.json`; its bounds and the independent resource, restoration, and health checks continue to govern validity.

A.2 Normalized Encoding and Candidate Identity

Let $z_j \in [0, 1]$ be normalized coordinate j , with native lower and upper bounds L_j and U_j . For an integer parameter, the domain contains $N_j = U_j - L_j + 1$ values, and decoding uses

$$x_j = L_j + \min(\lfloor z_j N_j \rfloor, N_j - 1). \quad (\text{A.1})$$

The upper cap ensures that $z_j = 1$ maps to U_j , while the other normalized coordinates partition into bins of equal width. Encoding a native integer returns the midpoint of its bin,

$$z_j = \frac{x_j - L_j + \frac{1}{2}}{N_j}. \quad (\text{A.2})$$

Consequently, encoding preserves the native configuration without claiming to recover the exact normalized point that originally generated it.

For a continuous parameter, decoding applies affine scaling followed by six-decimal quantization:

$$x_j = Q_6(L_j + z_j(U_j - L_j)), \quad z_j = \frac{x_j - L_j}{U_j - L_j}, \quad (\text{A.3})$$

where Q_6 denotes the implemented decimal quantization. The codec in `src/charmdb/v2/primary_design.py` validates coordinate bounds, parameter order, native bounds, and integer integrality. Because integer binning and continuous quantization can map different normalized proposals to the same settings, candidate identity is determined from the canonical decoded configuration. Duplicate checks therefore operate on the configuration that PostgreSQL receives, preserving the interpretation of a candidate budget as opportunities to evaluate distinct settings.

Appendix B

Protocol, Reproduction, and Evidence Traceability

B.1 Software and Experimental Specification

The software repository associated with this thesis is <https://github.com/asuta2/CHARM-DB> [32]. It organizes the implementation, automated tests, dependency specification, protocol manifests, and supporting documentation. Table B.1 identifies the entry points needed to connect the framework in Chapter 4 with the experimental design in Chapter 5. Paths in this table are relative to the repository root; directory names are source locators rather than separate experimental systems.

Table B.1: Software and protocol entry points

Purpose	Repository location
Protocol entry point	experiments/thesis/README.md
Methodology and benchmark profile	docs/methodology.md; docs/frozen-benchmark-profile.md
Primary design and continuation manifest	experiments/thesis/manifests/ primary-comparison.json; experiments/thesis/ manifests/primary-wave-b.json
Prefix and confirmatory protocols	experiments/thesis/manifests/multi-fidelity. json; experiments/thesis/manifests/ f4-confirmation.json
Combined-analysis preregistration	experiments/thesis/preregistration/ wave-b-final-analysis-preregistration.md
Implementation	src/charmdb/
Unit and integration tests	tests/unit/; tests/integration/
Dependencies and locked environment	pyproject.toml; uv.lock
Offline evidence audit	scripts/audit_thesis_closeout.py
Evidence assessment	experiments/thesis/provenance/thesis-closeout. md; experiments/thesis/provenance/ d072-evidence-audit.json

The primary seeds and logical and physical budgets are specified in Section 5.5 and Table 5.4. The manifests retain the internal scheduling division used to execute these observations, but the final analysis treats them as one five-seed experiment. Reconstructing the design there-

fore requires both primary manifests and the completed schedule ledger rather than interpreting an earlier manifest’s execution-status fields as the final experiment state.

The software environment is specified by the project metadata and dependency lock. The command `uv sync -extra dev -frozen` reconstructs the locked development environment from the preserved source tree. The unit suite is invoked with `uv run pytest tests/unit`. Integration tests additionally require their documented PostgreSQL services and configuration. These checks validate software behavior, whereas repeating the experiment also requires the pinned target image, resource envelope, baseline dataset, and measurement profile defined in Chapter 5. A successful test run alone does not reproduce the measured performance results.

B.2 Observation Lineage and Evidence Populations

Raw measurements and exported ledgers are stored separately from the source repository. In the research environment their root is `C:/CHARMDB-ARTIFACTS/v2`; the relative paths in Table B.2 identify the corresponding records within that tree. The separation permits large transaction logs and database artifacts to retain their own size and hash inventory without treating a repository checkout as a complete experimental archive. Reproduction of the reported results requires this evidence tree in addition to the software.

Table B.2: Evidence locations relative to the external evidence root

Evidence role	Relative location
File inventory and hashes	<code>evidence-sha256-manifest.json</code>
Ledger export index	<code>evidence-ledgers/index.json</code>
Primary schedule and candidates	<code>evidence-ledgers/primary-blocks.json</code> ; <code>evidence-ledgers/primary-runs.json</code>
Attempts and training history	<code>evidence-ledgers/primary-attempts.json</code> ; <code>evidence-ledgers/primary-training-lineage.json</code>
Restoration checks and timing	<code>evidence-ledgers/candidate-restore-ledger.json</code>
Raw trial artifacts	<code>raw/campaign-id/trial-id/</code>
Final primary analysis	<code>primary-wave-b/final-five-seed-analysis.json</code>
Final primary report	<code>primary-wave-b/report-final-five-seed/</code>
Retrospective and live prefixes	<code>multi-fidelity/phase-a/</code> ; <code>multi-fidelity/phase-b/</code>
Finalist confirmation	<code>f4-confirmation/f4-analysis.json</code> ; <code>f4-confirmation/report/</code>

A primary result is traced first to its scheduled run in `primary-runs.json`, which supplies the seed, method attribution, budget position, requested configuration, and proposal identity. Joining `primary_run_id` to `primary-attempts.json` identifies every attempt, including failures, and its trial identifier locates the raw measurement artifacts. The same trial identifier links restoration records, while the training-lineage export identifies observations eligible for an adaptive proposal. File paths and hashes then connect these records to the retained transaction logs and measurement summaries. This chain makes it possible to inspect the provenance of a reported objective without treating a rendered table as the original measurement.

The physical and logical populations must remain distinct during this reconstruction. Shared initialization is measured once but attributed to all three Bayesian histories; it contributes sixty

unique physical observations across five seeds. The primary attempts ledger contains 657 attempts for 655 completed physical observations, including two retained infrastructure failures. The retrospective prefix population contains 378 candidates from three seeds, the live demonstration contains thirty candidates and five controls, and confirmation contains twenty observations across four blocks. These secondary records retain their own analysis roles and do not increase the primary seed count or candidate budget.

B.3 Offline Verification and Reporting

The offline audit checks the external evidence inventory, reconciles primary runs and attempts, and regenerates the final report without connecting to PostgreSQL. The following invocation is executed from the implementation repository after the locked environment and source evidence are available:

```
uv run python scripts/audit_thesis_closeout.py
  --root C:/CHARMDB-ARTIFACTS/v2
  --output artifacts/reports/reproduction-audit
```

The three displayed lines form one command. The output directory must lie outside the source evidence tree, allowing regenerated files and corrections to be compared with the preserved inputs. This operation reproduces reporting from existing observations; it neither launches a benchmark nor repeats the physical experiment.

Verification relies on the complete file inventory and the final report index, with distinct identities for serialized files and canonical analysis payloads. The final analysis payload has SHA-256

```
769f5a0eabdebe272a0a8ae4b7578fd54
f72d4a95cd53822e1cb993466a36fb2
```

and the final report index file has SHA-256

```
97a37d469ca515a4fb4889f6548c8c42f
f33b85b1a41d6504dc2e4b551747fd2.
```

The digests exclude line breaks and final punctuation; the payload digest differs from a byte-level hash of its enclosing JSON file. The recorded full-tree audit verified 5,457 files and reproduced the seventeen-file report byte-identically.

The frozen report requires the safety-accounting correction used in Chapter 6: its original fields count first attempts as retries and can overcount shared failures. The audit supplies the corrected 655 physical slots, 657 attempts, and two retrieved slots, with two logical retry attributions per Bayesian method. Preserving the frozen report alongside this correction retains provenance without propagating its accounting defect.

The closeout record includes uncommitted implementation changes, so a repository URL or commit identifier alone cannot identify the reporting source snapshot. Reproduction requires the preserved working tree, dependency lock, protocol manifests, control-database backup, external evidence, and safety correction. Independent clean-machine replication and verification of a distributable archive remain unestablished. Access to software and deterministic report regeneration therefore remain distinct from replication of the experiment.

Bibliography

- [1] Van Aken, D., Pavlo, A., Gordon, G. J., and Zhang, B., “Automatic database management system tuning through large-scale machine learning”, in Proceedings of the 2017 ACM International Conference on Management of Data, 2017, pp. 1009–1024.
- [2] Shahriari, B., Swersky, K., Wang, Z., Adams, R. P., and de Freitas, N., “Taking the human out of the loop: A review of bayesian optimization”, Proceedings of the IEEE, Vol. 104, No. 1, 2016, pp. 148–175.
- [3] Duan, S., Thummala, V., and Babu, S., “Tuning database configuration parameters with iTuned”, Proceedings of the VLDB Endowment, Vol. 2, No. 1, 2009, pp. 1246–1257.
- [4] Zhang, X., Chang, Z., Li, Y., Wu, H., Tan, J., Li, F., and Cui, B., “Facilitating database tuning with hyper-parameter optimization: A comprehensive experimental evaluation”, Proceedings of the VLDB Endowment, Vol. 15, No. 9, 2022, pp. 1808–1821.
- [5] Jones, D. R., Schonlau, M., and Welch, W. J., “Efficient global optimization of expensive black-box functions”, Journal of Global Optimization, Vol. 13, No. 4, 1998, pp. 455–492.
- [6] Cereda, S., Valladares, S., Cremonesi, P., and Doni, S., “CGPTuner: A contextual gaussian process bandit approach for the automatic tuning of it configurations under varying workload conditions”, Proceedings of the VLDB Endowment, Vol. 14, No. 8, 2021, pp. 1401–1413.
- [7] PostgreSQL Global Development Group, PostgreSQL 18.1 Documentation, PostgreSQL Global Development Group, 2025, Available: <https://www.postgresql.org/docs/18/>
- [8] Dean, J., and Barroso, L. A., “The tail at scale”, Communications of the ACM, Vol. 56, No. 2, 2013, pp. 74–80.
- [9] Hutter, F., Hoos, H. H., and Leyton-Brown, K., “Sequential model-based optimization for general algorithm configuration”, in Learning and Intelligent Optimization, ser. Lecture Notes in Computer Science, Vol. 6683. Springer, 2011, pp. 507–523.
- [10] Rasmussen, C. E., and Williams, C. K. I., Gaussian Processes for Machine Learning. Cambridge, MA, USA: MIT Press, 2006.
- [11] Eriksson, D., Pearce, M., Gardner, J., Turner, R. D., and Poloczek, M., “Scalable global optimization via local bayesian optimization”, in Advances in Neural Information Processing Systems, Vol. 32, 2019.
- [12] Snoek, J., Larochelle, H., and Adams, R. P., “Practical bayesian optimization of machine learning algorithms”, in Advances in Neural Information Processing Systems, Vol. 25, 2012, pp. 2951–2959.

- [13] Ament, S., Daulton, S., Eriksson, D., Balandat, M., and Bakshy, E., “Unexpected improvements to expected improvement for bayesian optimization”, in *Advances in Neural Information Processing Systems*, Vol. 36, 2023.
- [14] Knowles, J., “ParEGO: A hybrid algorithm with on-line landscape approximation for expensive multiobjective optimization problems”, *IEEE Transactions on Evolutionary Computation*, Vol. 10, No. 1, 2006, pp. 50–66.
- [15] Paria, B., Kandasamy, K., and Póczos, B., “A flexible framework for multi-objective bayesian optimization using random scalarizations”, in *Proceedings of the 35th Conference on Uncertainty in Artificial Intelligence*, ser. *Proceedings of Machine Learning Research*, Vol. 115, 2020, pp. 766–776.
- [16] Daulton, S., Balandat, M., and Bakshy, E., “Differentiable expected hypervolume improvement for parallel multi-objective bayesian optimization”, in *Advances in Neural Information Processing Systems*, Vol. 33, 2020.
- [17] Daulton, S., Balandat, M., and Bakshy, E., “Parallel bayesian optimization of multiple noisy objectives with expected hypervolume improvement”, in *Advances in Neural Information Processing Systems*, Vol. 34, 2021.
- [18] Kanellis, K., Alagappan, R., and Venkataraman, S., “Too many knobs to tune? towards faster database tuning by pre-selecting important knobs”, in *12th USENIX Workshop on Hot Topics in Storage and File Systems*. USENIX Association, 2020, Available: <https://www.usenix.org/conference/hotstorage20/presentation/kanellis>
- [19] Kanellis, K., Ding, C., Kroth, B., Müller, A., Curino, C., and Venkataraman, S., “LlamaTune: Sample-efficient dbms configuration tuning”, *Proceedings of the VLDB Endowment*, Vol. 15, No. 11, 2022, pp. 2953–2965.
- [20] Gardner, J. R., Kusner, M. J., Xu, Z., Weinberger, K. Q., and Cunningham, J. P., “Bayesian optimization with inequality constraints”, in *Proceedings of the 31st International Conference on Machine Learning*, ser. *Proceedings of Machine Learning Research*, Vol. 32, No. 2, 2014, pp. 937–945.
- [21] Kandasamy, K., Dasarathy, G., Schneider, J., and Póczos, B., “Multi-fidelity bayesian optimisation with continuous approximations”, in *Proceedings of the 34th International Conference on Machine Learning*, ser. *Proceedings of Machine Learning Research*, Vol. 70, 2017, pp. 1799–1808.
- [22] Weikum, G., Mönkeberg, A., Hasse, C., and Zabback, P., “Self-tuning database technology and information services: From wishful thinking to viable engineering”, in *Proceedings of the 28th International Conference on Very Large Data Bases*. Morgan Kaufmann, 2002, pp. 20–31.
- [23] Chaudhuri, S., and Narasayya, V. R., “Self-tuning database systems: A decade of progress”, in *Proceedings of the 33rd International Conference on Very Large Data Bases*, 2007, pp. 3–14.
- [24] Zhu, Y., Liu, J., Guo, M., Bao, Y., Ma, W., Liu, Z., Song, K., and Yang, Y., “BestConfig: Tapping the performance potential of systems via automatic configuration tuning”, in *Proceedings of the 2017 Symposium on Cloud Computing*, 2017, pp. 338–350.

- [25] Van Aken, D., Yang, D., Brillard, S., Fiorino, A., Zhang, B., Bilien, C., and Pavlo, A., “An inquiry into machine learning-based automatic configuration tuning services on real-world database management systems”, *Proceedings of the VLDB Endowment*, Vol. 14, No. 7, 2021, pp. 1241–1253.
- [26] Zhang, X., Wu, H., Chang, Z., Jin, S., Tan, J., Li, F., Zhang, T., and Cui, B., “ResTune: Resource oriented tuning boosted by meta-learning for cloud databases”, in *Proceedings of the 2021 International Conference on Management of Data*, 2021, pp. 2102–2114.
- [27] Fekry, A., Carata, L., Pasquier, T., Rice, A., and Hopper, A., “To tune or not to tune? in search of optimal configurations for data analytics”, in *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2020, pp. 2494–2504.
- [28] Zhang, J., Liu, Y., Zhou, K., Li, G., Xiao, Z., Cheng, B., Xing, J., Wang, Y., Cheng, T., Liu, L., Ran, M., and Li, Z., “An end-to-end automatic cloud database tuning system using deep reinforcement learning”, in *Proceedings of the 2019 International Conference on Management of Data*, 2019, pp. 415–432.
- [29] Li, G., Zhou, X., Li, S., and Gao, B., “QTune: A query-aware database tuning system with deep reinforcement learning”, *Proceedings of the VLDB Endowment*, Vol. 12, No. 12, 2019, pp. 2118–2130.
- [30] Balandat, M., Karrer, B., Jiang, D. R., Daulton, S., Letham, B., Wilson, A. G., and Bakshy, E., “BoTorch: A framework for efficient monte-carlo bayesian optimization”, in *Advances in Neural Information Processing Systems*, Vol. 33, 2020, pp. 21 524–21 538.
- [31] BoTorch Contributors, “BoTorch 0.17.2: Model and fitting source code”, <https://github.com/meta-pytorch/botorch/tree/v0.17.2>, 2026.
- [32] Šuta, A., “CHARM-DB: Software repository for database optimization via machine learning”, GitHub repository, Available: <https://github.com/asuta2/CHARM-DB> 2026.
- [33] Efron, B., “Bootstrap methods: Another look at the jackknife”, *The Annals of Statistics*, Vol. 7, No. 1, 1979, pp. 1–26.
- [34] Cliff, N., “Dominance statistics: Ordinal analyses to answer ordinal questions”, *Psychological Bulletin*, Vol. 114, No. 3, 1993, pp. 494–509.
- [35] Holm, S., “A simple sequentially rejective multiple test procedure”, *Scandinavian Journal of Statistics*, Vol. 6, No. 2, 1979, pp. 65–70, Available: <https://www.jstor.org/stable/4615733>